FeatureSAGE: Stable Attribute Group Editing With Integrated StyleFeatureEditor Encoder for Enhanced Few-shot Image Generation

Joshua Libando[1*] and Sam Joash V. Antonio[1]

[1*]Department of Computer Science, College of Computing and Information Sciences, Caraga State University – Main Campus, Butuan City, Philippines.

Abstract

Few-shot image generation remains challenging because limited training data can lead to mode collapse, degraded image quality, and unstable semantic edits. Traditional Generative Adversarial Network (GAN) inversion methods also struggle to balance reconstruction fidelity with editability, particularly in attribute group editing frameworks where semantic consistency and fine-grained detail preservation are required. The Stable Attribute Group Editing (SAGE) framework addresses few-shot generation through latent attribute editing, but it relies on the pixel2style2pixel (pSp) encoder, which can produce entangled latent codes and lose local reconstruction details. To address this limitation, this study proposes FeatureSAGE, an enhanced few-shot image generation framework that integrates the StyleFeatureEditor (SFE) Inverter and Feature Editor into the SAGE architecture. The proposed framework represents images using both $\boldsymbol{W}^+$ latent codes and intermediate feature tensors, enabling latent-space editing while preserving feature-level details. Experimental evaluation on the FUNIT Animal Faces and 102 Flowers datasets at 1024×1024 resolution shows that FeatureSAGE improves distributional alignment over the AGE and SAGE baselines. On the Flowers dataset, FeatureSAGE obtains an FID of 45.60, corresponding to 15.8% and 25.9% improvements over SAGE and AGE, respectively, while reporting an LPIPS of 0.4209. On the Animal Faces dataset, FeatureSAGE achieves an FID of 49.60, corresponding to 18.6% and 20.2% improvements over SAGE and AGE, respectively, with an LPIPS of 0.4811. These results indicate that integrating feature-level editing with improved latent inversion strengthens few-shot attribute group editing without changing the underlying StyleGAN2 generator.

Keywords: Few-shot image generation, GAN inversion, StyleGAN2, StyleFeatureEditor, attribute group editing

1 Introduction

1.1 Background of the Study

Generative Adversarial Networks (GANs) have become a foundational framework for high-fidelity image generation. In a GAN, a generator and a discriminator are trained in a minimax game: the generator learns to produce synthetic images that fool the discriminator into classifying them as real, while the discriminator learns to distinguish generated images from real ones [15]. This adversarial process has enabled GANs to achieve remarkable realism in generated images (e.g. faces, animals) across many domains. In particular, the StyleGAN family of architectures [22] introduced a novel style-based generator that has set new standards for image quality. StyleGAN maps an input latent code z through a learned mapping network into an intermediate latent w in a space called W. Each layer of the generator is then modulated by w via adaptive instance normalization (AdaIN), and stochastic noise inputs add fine detail. This style-based design yields an intermediate latent space W that is significantly more disentangled than the original input space Z. As a result, controlled manipulation of individual semantic attributes (e.g. hair color, pose) is possible by adjusting w or the per-layer style codes. Subsequent improvements (StyleGAN2, StyleGAN3) refined the architecture and training, further boosting image fidelity and easing inversion (mapping real images back to latent codes).

StyleGAN's latent representations come in several forms. The basic mapping from Z to W produces a single 512-D style code w per image. An extended latent space W^+ is also used, where a separate w vector is specified for each of the 18 synthesis layers. An even higher-dimensional feature space F can be formed by concatenating intermediate style and feature tensors (sometimes denoted F_k), capturing spatial details in the generator. Recent work has highlighted a trade-off across these spaces: moving from W to W^+ to F increases reconstruction fidelity but reduces editability [24] . In other words, encoding an image in the low-dimensional W space yields highly edit-friendly representations but may lose fine detail, whereas the high-dimensional W^+ or F spaces can reconstruct details at the cost of some entanglement and overfitting. StyleGAN2 in particular has shown that inversion in W^+ benefits from regularization (e.g. path-length regularization) to improve reconstructability and make projected images easier to attribute to latent codes [22].

To edit real images with a GAN, one must first perform GAN inversion: find a latent code that generates (nearly) the given image. This is a well-studied but challenging problem [4, 32]. Early inversion methods used optimization (iteratively updating z to minimize reconstruction error), which can achieve high fidelity but is slow and may produce out-of-distribution latents. Encoder-based methods train a neural network to predict the latent directly, enabling real-time inversion at the expense of some quality. For StyleGAN, [32], proposed Pixel2Style2Pixel (pSp): a feed-forward encoder that maps any input image to a sequence of style vectors in W^+ (one w per layer). The pSp encoder can invert faces (or other images) into W^+ accurately without test-time optimization, greatly simplifying downstream editing. Generally, as noted by [32], "inversion methods either directly optimize the latent vector to minimize the error for

the given image". Encoder approaches like pSp offer fast inference, while optimization-based methods remain stronger on pure reconstruction quality. In either case, the goal of GAN inversion is to find an internal representation that contains as much image detail as possible and still allows semantic edits [4].

Once a real image is embedded in StyleGAN's latent space, attribute editing becomes possible. It has been observed that semantic attributes (e.g. "smile," "age," "eyeglasses") correspond to linear directions or subspaces in the latent space. For example, [35] showed that well-trained StyleGAN latent codes encode a disentangled representation under linear transformations. In practice, one can move the latent w of an image along a learned direction Δw to change a particular attribute (e.g. $w + \alpha \Delta w$) [12]. Many works have exploited this: e.g. Linear SVM or PCA over latent codes can find directions for "smile" vs "no smile," or for pose changes This allows controlled editing via vector arithmetic. Importantly, style-based latents like W and W^+ are far less entangled than raw Z, so edits tend to be more semantically meaningful [22, 35]. Still, some entanglement remains changing one attribute may inadvertently affect others. Recent methods (e.g. using subspace projection or network-based attribute classifiers) seek to further disentangle attributes. Moreover, higher-dimensional latent spaces (like W^+ or F) can encode finer identity details, enabling extremely high-quality reconstructions. [4] leverage this by introducing a StyleFeatureEditor: an encoder that predicts both a W^+ code w and an intermediate feature tensor F_k. By editing in both w and F_k, they reconstruct and preserve finer image details even under heavy edits. In summary, StyleGAN latent spaces provide a powerful substrate for attribute editing, but the choice of W vs W^+ vs F presents a trade-off between edit stability and detail, and sophisticated encoders are needed to balance both.

A parallel line of research addresses few-shot image generation and editing. Unlike standard GAN training (which requires large datasets), few-shot generation aims to adapt a model to a new category or attributes given only a handful of examples. Meta-learning approaches have been applied here: for instance, the FIGR model [9] meta-trains a GAN with the Reptile algorithm so it can generate new classes from as few as 4 examples. Other works like FUNIT [26] tackle few-shot domain translation, using image encoders to condition style generation on a few references. However, generating realistic variation from very limited data remains challenging. In particular, attribute group editing has emerged as a promising strategy for few-shot GANs. The idea is to collect many examples of known categories to learn generic attribute directions, then apply those directions to few-shot target categories. AGE learns a dictionary of "category-irrelevant" attribute vectors (e.g. pose, color) from seen classes and "category-relevant" class embeddings (mean latent per class) [12]. At inference, given one or a few examples of an unseen category, AGE embeds them into W^+ (using pSp) and manipulates those latents by combining the learned attribute directions. Similarly,[44] propose a Transformer-style approach with modules for learning discrete attribute codebooks and prompt-driven editing to handle diverse unseen classes. These methods show that by factorizing and reusing attribute vectors, a pre-trained GAN can generate plausible images of new categories without full retraining [12]. Nonetheless, they often assume large amounts of labeled data for the seen categories

and focus on one attribute at a time or on broad class attributes (like "breed of dog" vs "background color of bird"), rather than coordinated multi-attribute control.

Despite these advances, important gaps remain. Most GAN inversion and editing methods have focused on single-attribute manipulations or on scenarios with abundant data. Achieving stable, coordinated multi-attribute edits is still difficult, especially under few-shot conditions. For example, existing StyleGAN inversion methods either excel at reconstruction but produce entangled edits, or vice versa [24, 32]. Moreover, according to [13] , "class-inconsistency" is a common-problem GAN-generated images for downstream classification. Class inconsistency happens when the generative model corrupts some minor but critical information versus real pictures of a certain category [13]. Few-shot models like AGE and TAGE demonstrate that group editing is possible, but they often do not explicitly ensure fidelity or identity preservation when combining many edits. Moreover, generation quality in few-shot setups can degrade due to mode collapse or identity drift. In face editing, for instance, we desire that an edited face remains clearly the same person (high identity similarity) while attributes like expression or hair style change. Conventional metrics like Frechet Inception Distance (FID) and LPIPS capture distribution quality and perceptual similarity [22, 42].

We proposed that achieving improved and stable group-wise attribute editing under few-shot conditions required an integrated approach. FeatureSAGE is designed to address this by replacing the standard pSp encoder of SAGE with the StyleFeatureEditor Inverter and Feature Editor. The integrated encoder (inspired by StyleFeatureEditor) jointly predicts a W^+ latent w and a feature tensor F, aiming for high-fidelity inversion as well as explicit control of feature-level detail. This architecture is intended to preserve identity and image realism (as measured by identity similarity and low FID/LPIPS) while achieving disentangled, controlled modifications of target attributes. In summary, the background works on GANs, StyleGAN latent editing, and few-shot group editing together motivate FeatureSAGE's goal: to provide stable, coordinated multi-attribute control in the latent space of a pre-trained GAN, even with only limited example images, without sacrificing image quality or attribute disentanglement.

1.2 Statement of the Problem

This study examines the main limitations of the current SAGE approach, specifically the use of the Pixel2Style2Pixel (pSp) encoder, which tends to yield latently entangled codes and diminished control over semantic edits. The limitations reduce the creation of semantic-consistent, high-fidelity images from few-shot instances. While GAN-produced images are visually realistic, loss of fine details from inversion heavily degrades downstream performance for classification. In approaches such as SAGE, using the reconstructed image for training lowers accuracy because of the mismatch between the latent embeddings and the original information, as well as the lack of fidelity in the reconstructions. The loss of information is carried over during the editing process, diminishing the performance of the generative augmentation. Enhanced inversion would increase both editability of the image and downstream task performance. Specifically, the study seeks to answer the following research questions:

1. Can integrating StyleFeatureEditor inverter and Feature Editor improve latent inversion fidelity and preserve image details for style-based editing within the SAGE framework?
2. Do these modifications influence the consistency, diversity, and quality of images generated in the few-shot setting, particularly for novel classes in datasets Flowers and Animal Faces?
3. How well does FeatureSAGE perform in terms of editing reliability and disentanglement in few-shot scenarios?
4. How does FeatureSAGE compare to existing editing based few shot image generation methods, AGE and SAGE, in terms of edit stability, identity preservation, and overall image quality?

1.3 Objectives of the Study

The objective of this study is to develop and evaluate FeatureSAGE, a reliable few-shot image generator framework that improves upon the SAGE framework by integrating a StyleFeatureEditor Inverter and Feature Editor

Specifically , this study's object is as follows:

1. Integrate StyleFeatureEditor into the SAGE framework by incorporating the inverter and feature editor pair to improve generated image quality. This aims to make a framework variant of SAGE that can be used for a more downstream (e.g. Data Augmentation) task via high reconstructability and editability without the usual tradeoff.
2. Evaluate reconstruction quality of the inverted images of Animal Faces and Flowers using FID and LPIPS.
3. Benchmark FeatureSAGE against baseline methods, specifically SAGE and AGE, to demonstrate improvements in edit stability, identity preservation, and overall image quality.

1.4 Significance of the Study

This study contributes to the few-shot image generation domain by enhancing the editability and interpretability of latent codes in the SAGE framework. By integrating the StyleFeatureEditor's encoder and feature editor, the proposed FeatureSAGE framework improves stability and consistency in attribute group manipulation under data-scarce cases which addresses the key limitation of the existing few-shot generative models.

Aside from methodological contributions, the proposed approach also has practical relevance and has several downstream applications. In fine-grained visual recognition, FeatureSAGE can generate identity-preserving variations of an animal or plant from a very few dataset, which helps train classifiers for species or breed identification in environments where collecting large dataset is impossible or difficult to acquire. In biometric work that focuses on identity, such as forensic image analysis and identity-based data augmentation, FeatureSAGE can generate multiple edited versions of an image while keeping the same person's identity. This makes the findings more realistic and more reliable. In medical and scientific imaging, researchers often have small datasets

because privacy rules limit sharing, and gathering new images can be expensive. In some cases, the condition or disease is rare.FeatureSAGE generates realistic images that can serve as a synthetic dataset for training and validating machine learning models.

This study extends prior work on attribute group editing and GAN latent spaces, and it gives clear guidance for building data-efficient image generation systems that can support downstream applications that need consistent, reliable image synthesis.

1.5 Scope and Limitation

This study is limited to few-shot image generation tasks using the StyleGAN2 generator. It specifically focuses on improving the reconstruction and editability of the model by integrating the StyleFeatureEditor Encoder and utilizing the Feature Editing Module into the framework of SAGE.

The datasets used in this study are restricted to Animal Faces and Flowers which offer diverse but structured categories suitable for evaluating attribute preservation and class coherence. The study does not explore optimization-based inversion techniques or apply to GAN architectures other than StyleGAN2. Results and conclusions are therefore constrained to the SAGE pipeline with these enhancements.

2 Review of Related Literature

2.1 Generative Adversarial Networks

Generative Adversarial Networks (GANs) is a novel framework for training generative models via an adversarial process which was first introduced by [15]. In the paper , "Generative Adversarial Nets" , a generator network G produces samples aimed at mimicking the data distribution , while the discriminator D estimates the probability of whether an input is real or generated [15]. In other words, GAN framework basically corresponds to a minimax two-player game where , a generator (G) creates samples that are intended to come from the same distribution as the training data, while the discriminator D examines the probability of the the samples provided by G being real or fake [14].

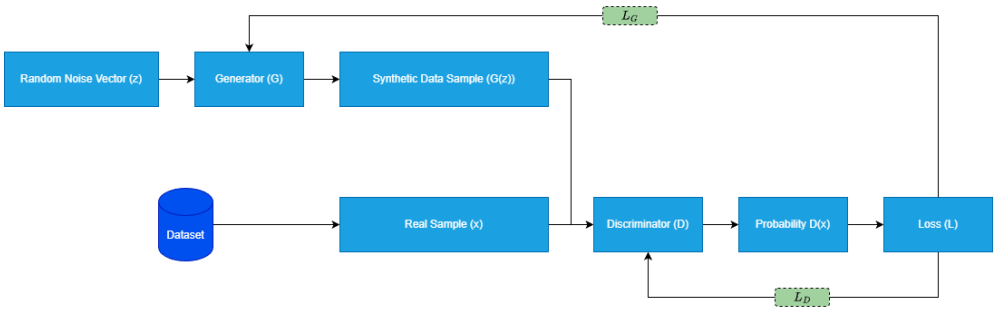

Fig. 1 *Generative Adversarial Networks (GANs) Architecture*

As shown in Figure 1 the generator G takes as input a random noise vector $z \sim p_z$ (usually drawn from a simple distribution such as uniform or Gaussian) and outputs a synthetic data sample $G(z)$. The goal of G is to map the "latent" noise distribution into the data sample $G(z)$ is as realistic as possible. The discriminator D takes a data sample x as input and outputs a probability $D(x)$ estimating the likelihood that x is a real sample rather than one produced by G [10, 11, 33].

The adversarial training process in GANs is governed by a minimax optimization. Formally, [15] define the value function:

$$V(D,G) = \mathbb{E}_{x\sim p_{\text{data}}(x)}[\log D(x)] + \mathbb{E}_{z\sim p_z(z)}[\log(1 - D(G(z)))]$$

and train D and G in a two-player game:

$$\min_G \max_D V(D,G)$$

D and G are alternatively updated via minibatch stochastic gradient descent during the GAN training process. D is trained to maximize the probability of predicting whether an input from both training example and sample from G are real or fake, while G is trained simultaneously to minimize $\log(1 - D(G(z)))$ [15]. In reality, GAN training can be delicate: if G becomes too strong or D too weak , gradients can vanish, and if D is too strong early on , G may recieve no learning signal(saturating loss). [15] suggest a non-saturating heuristic for G (maximizing $\log D(G(z))$) instead of minimizing $\log(1 - D(G(z)))$ to provide stronger gradients in practice.

On its first introduction , GANs have brought about advantages compared to other generative models at the time (e.g. Variational Auto-encoders(VAE)) like sharper images, configurable sizes, richer search space, and a veratility as it can support different generator networks. However it also has its disadvatages like : mode collapse, vanishing gradients , and instability [10].

In the years following the initial discovery of Generative Adversarial Networks (GANs), there have been numerous improvements made to their architecture that have resulted in better image quality, more stable training, and expanded capabilities. Early research highlighted the importance of convolutional structures for successful GAN performance. One influential contribution was made by [32], who introduced the Deep Convolutional GAN (DCGAN) and a set of practical design recommendations for building effective convolutional GANs. Instead of employing traditional pooling or unpooling layers, DCGAN utilized strided convolutions to decrease spatial resolution and transposed convolutions to increase it. This allowed both G and D to learn spatial features more effectively and led to outputs that were more realistic [14]. They also recommended batch normalization in almost all layers of both G and D, excluding the output and input layers respectively, which greatly stabilized training and facilitated learning proper scales.

These guidelines enabled DCGANs to synthesize high-resolution images in "one shot" and learn disentangled latent representations that enable simple vector arithmetic for changing semantic attributes of the output. DCGAN also popularized the use

of Adam as an optimizer for GANs [14]. In conclusion, this research provides a comprehensive overview of how convolutional structures can improve the performance of GANs, and how practical design recommendations such as strided convolutions, transposed convolutions, batch normalization, and Adam can enable these improvements. By following these guidelines, it is possible to build high-quality image synthesis models that learn complex spatial relationships in images and produce realistic outputs. In particular, DCGANs are able to synthesize high-resolution images in "one shot" and learn disentangled latent representations that allow for simple vector arithmetic for changing semantic attributes in the output, as well as enable more flexible model configurations by allowing the use of a different optimizer, such as Adam, which has been shown to be effective in training GANs. Furthermore, DCGANs popularized the use of batch normalization across all layers in both the generator and discriminator networks, excluding the output and input layers respectively, which greatly stabilized training and facilitated learning proper scales. These findings highlight the importance of convolutional structures in improving the performance of GANs and provide a comprehensive overview of how practical design recommendations can be used to build high-quality image synthesis models that learn complex spatial relationships in images and produce realistic outputs.

While StyleGAN and StyleGAN2 have achieved impressive results for image realism, they still face some challenges. The main limitation of StyleGAN lies in its inability to accurately imitate real-world images and in the fact that it cannot directly manipulate the latent space within the network. Additionally, even when inversion is implemented, the resulting images may not always capture identity or be structurally accurate. Despite these limitations, StyleGAN has shown promise in some unstructured scenes and faces. However, as [3] observed, StyleGAN struggles with domains that do not exhibit strong structure and may even replicate demographic biases in the training set. Finally, while StyleGAN can generate images at high resolution with ease, it requires extensive GPU resources to train on high-resolution data. Moreover, careful tuning of the architecture is also necessary to ensure fairness and accuracy. Overall, while StyleGAN has achieved significant progress for image realism, there are still challenges that need to be addressed in order to fully utilize its potential for creative and sophisticated use cases.

2.2 StyleGAN and StyleGAN2

Early GANs suffered from instability and mode collapse, which [20] addressed by Progressive GANs (PGGAN): training begins at low resolution and gradually adds layers as resolution increases [27]. Building on this, StyleGAN [21] proposed a novel style-based generator architecture that achieved state-of-the-art image quality and disentangled latent factors. StyleGAN is a variant of GAN that builds on its framework by radically redesigning the generator.

Instead of feeding the latent code $\mathbf{z}$ directly into the generator, StyleGAN first maps $\mathbf{z} \sim \mathcal{N}(0, I)$ to an intermediate latent $\mathbf{w} = f(\mathbf{z})$ using an 8-layer fully-connected network [21]. This $\mathbf{w}$ lives in a new "style" space $\mathcal{W}$ that is learned to be less entangled than the original latent space. Learned affine transforms then convert $\mathbf{w}$ into per-layer style parameters (y_s, y_b) that modulate each convolution via Adaptive Instance

Normalization (AdaIN) [21, 22]. Concretely, for each feature map x_i in a convolutional layer, StyleGAN applies:

$$\text{AdaIN}(x_i, y) = y_{s,i} \left(\frac{x_i - \mu(x_i)}{\sigma(x_i)} \right) + y_{b,i}$$

, where y_s and y_b (scalars for channel i) are derived from $\mathbf{w}$. In practice, StyleGAN's mapping network f (an eight-layer MLP) produces $\mathbf{w}$, and then each convolutional layer is modulated by the corresponding (y_s, y_b) pair. This allows scale-specific control of features: coarse spatial styles (e.g. pose or overall shape) are set by early layers, while fine-grained details (e.g. color, texture) are set by later layers [21, 27]. Notably, the mapping network and style modulation enable style-mixing: two latent codes can be injected at different depths, blending coarse features from one code with fine details from another [27].

Importantly, StyleGAN replaces the standard random input with a learned constant 4x4x512 tensor, and all variation comes from the style vectors and added noise. At each convolutional layer, StyleGAN also injects a random noise map (one per pixel) which is added to the features with learned per-channel weights. This stochastic noise produces high-frequency variation (e.g. hair wisps, freckles) without affecting global attributes. Together, these design choices yield unsupervised disentanglement of high-level attributes from stochastic detail: [21] observe that their style-based generator "leads to an automatically learned, unsupervised separation of high-level attributes (e.g., pose and identity) and stochastic variation in the generated images". In experiments, StyleGAN achieves state-of-the-art image fidelity (low Frechet Inception Distance) and more linear, less entangled latent factors than conventional GANs.

The architectural differences from a standard GAN can be summarized as:

- Intermediate latent space and mapping network: StyleGAN learns a mapping $f : \mathbf{z} \to \mathbf{w}$ (eight-layer MLP) so that $\mathbf{w}$ in space $\mathcal{W}$ is disentangled
- Style modulation via AdaIN: Style vectors from $\mathbf{w}$ provide per-layer scale/bias that modulate each convolution by AdaIN, as in the formula above.
- Noise injection: Learned Gaussian noise is added to each layer to control stochastic details
- Learned constant input: The generator begins from a fixed learned 4x4 tensor instead of random noise
- Progressive growing: StyleGAN adopts the progressive training strategy from ProGAN [Karras et al., 2018], starting at 4x4 resolution and gradually adding layers to reach high resolution [20].

StyleGAN2 [22] builds directly on StyleGAN but redesigns the generator to eliminate artifacts and boost quality. The key insight is that the original AdaIN normalization can destroy meaningful signal: normalizing each feature map breaks global correlations needed for coherent details. Therefore, StyleGAN2 "bakes" the style modulation into the convolution weights and replaces the per-feature normalization with a weight demodulation operation. In practice, this works as follows: let w_{ijk} be the convolution weight connecting input channel i to output channel j (at spatial position k). Given a style scale s_i (from $\mathbf{w}$) for channel i, StyleGAN2 first multiplies w_{ijk} by s_i

(modulation), yielding $w'_{ijk} = s_i w_{ijk}$. It then computes the output standard deviation σ_j of the convolution and demodulates by dividing by σ_j:

$$w''_{i,s,j,k} = \frac{w'_{ijk}}{\sqrt{\sum_{i'}(w'_{ijk})^2 + \epsilon}}$$

. This eliminates the need for explicit instance normalization after the convolution. The result is that each convolutional layer can apply the style s_i directly to its weights, with the demodulation automatically preserving unit variance. This modification removes the "blob" artifacts common in StyleGAN outputs and leads to crisper, more natural images.

Training in StyleGAN2 takes a different path. Instead of slowly increasing complexity, the model uses a set structure right away. Skip connections and residual blocks link features across scales, much like in MSG-GAN or U-Net designs. This setup keeps signal flow strong throughout training. Without needing to add layers over time, stability improves naturally. The approach skips the extra burden of gradual expansion. Yet, key elements remain - style mapping and noise inputs stay part of the process. These are simply embedded into the updated generator framework. Stability meets continuity here. Not every change means replacing what worked. Another finding from [22] shows that applying path length regularization to the mapping network helps balance how shifts in W affect visual output. Because of this adjustment, movements through latent space translate more consistently into noticeable image differences. The result is a smoother connection between codes and images, easier reversal from image back to code, better separation of features, along with improved editing control.

The architectural improvements in StyleGAN2 can be summarized as:

- Weight demodulation instead of AdaIN: Styles modulate convolution weights and are normalized in-weight, removing all explicit feature-map normalization.
- No instance normalization layers: By design, the generator no longer uses AdaIN or other instance/batch norms; all normalization is implicit in weight demodulation
- Elimination of progressive growing: The network is trained at fixed full resolution, using skip/residual connections to preserve stability
- Path length regularization: A regularizer on the mapping network encourages a smooth, linearized latent space.

These changes yield a substantial quality gain. For example, [21] report that fully updated StyleGAN2 (no progressive growing, weight demodulation, path regularization) achieves a Frechet Inception Distance of about 3.3 on FFHQ, compared to 4.4 for the original StyleGAN baseline.

Though StyleGAN and its successor deliver remarkable visual fidelity, certain constraints remain evident. Generated outputs along with manipulation options depend strictly on what the model learned during training. Control through latent space works solely within the boundaries of synthetic imagery the system supports; actual photographs require conversion into domain via GAN inversion prior to adjustments [3]. Yet perfect preservation of subject traits isn't always achieved post-inversion. What

makes face images easier for these models becomes a hurdle elsewhere - disorganized settings confuse their output. Bermano and team noted back in 2022 how StyleGAN falters when patterns lack clear repetition. Problems show up beyond performance too, since learned behaviors include social imbalances present in source data. Training runs expose another layer of difficulty: high-res workloads need powerful graphics processors, precise adjustments. Complexity piles up, even if results look effortless.

2.3 The Latent Space

Early GANs drew a latent vector z from a simple distribution and fed it through a generator, but the resulting latent space was largely entangled and hard to control. InfoGAN by [7] introduced an information-maximizing objective to encourage disentanglement, yielding interpretable latent factors. The progressive GAN (PGGAN) of [20] improved image quality via multiresolution training but still used a single latent code at the input. A major shift occurred with StyleGAN [21], which used a learned mapping network to transform the input z into an intermediate latent W space This mapping ,via 8 fully-connected layers, "unfolds" the simple z distribution into W, aligning it to the generator's feature distribution and enabling axis-parallel edits[27]. StyleGAN also injects this W code and a random per-layer noise at each convolutional layer, allowing layer-wise control of coarse-to-fine image attributes.

StyleGAN2 built on this by replacing Adaptive Instance Normalization (Ada in) with weight modulation and demodulation; it further regularized the latent mapping to smooth out the generator's response to latent shifts [22]. This made the latent-to-image mapping more uniform and significantly easier to invert, meaning real images could be projected more reliably into W. Importantly, StyleGAN2 popularized the use of an extended latent space W^+, in which a separate 512-dim w vector is supplied to each of the 18 generator layers. In W^+, each layer can have its own style code, greatly increasing representational power: for a 1024x1024 model, W^+ is 18x512-dim. However, W^+ vectors do not come from the learned W distribution, so naive sampling can produce realistic "face-like" images but also novel patterns that are still face-like (eg. Aliens) [27]. W^+ is used mostly for reconstruction/inversion and fine edits, whereas W remains the base latent space for smooth, in-distribution sampling. Finally, StyleGAN2's style code w is further transformed, once per layer, into a channel-wise affine vector s in Style Space S, which directly scales convolutional feature maps [27]. In StyleGAN2 , each channel's style parameter in S is a single scalar, controlling only scale (no bias). Thus StyleGAN2 introduced multiple nested latent spaces: the input Z, the mapped W, the layered W^+, and the channel-wise S.

The StyleGAN architecture encourages disentanglement by design. [22] observed that W becomes "much closer to the required feature distribution" than z, so that semantic edits align with simple directions. In StyleGAN the W space yields more disentangled representations than the original Z space, making long-range manipulations (e.g. aging) more robust [35]. InterFaceGAN explicitly exploits this: it identifies linear hyperplanes in W corresponding to binary face attributes (smile, age, eyeglasses, etc.) and shows that moving w across these decision boundaries toggles the attribute. In StyleGAN's W space, these attributes become nearly disentangled after a linear transformation, unlike in Z.

Unsupervised methods have since discovered many such directions. GANSpace applies Principal Component Analysis (PCA) to StyleGAN2 latent codes and finds that principal components correspond to high-level edits (pose, color, environment) [17]. emarkably, GANSpace shows that a single global PCA direction, when applied at specific layers ("layer-wise decomposition"), can control attributes from coarse (camera angle) to fine (facial features) without labels. Similarly, [38] developed an unsupervised optimization to extract interpretable latent directions from any GAN; they report non-trivial findings (e.g. a "background removal" direction) without any supervision. Semantic Factorization (SeFa) by [34] provides a complementary approach by performing a closed-form eigen-decomposition of the pretrained generator's weight matrices, SeFa factors the W space into semantically meaningful directions (necklace on/off, lighting, etc.) without training or samples. Empirically, SeFa's directions often match or exceed those from supervised methods (e.g. InterFaceGAN) in manipulating specific attributes.

The disentangled latent structure of StyleGAN2 enables rich image manipulation. A common paradigm is latent traversal: given a latent code w, one perturbs it along a semantic direction and observes the image change. This can be simple as linear arithmetic like how InterfaceGAN uses $z_{edit} = z + \alpha n$ for single attribute manipulation. More advanced controllers have been developed. StyleFlow (Abdal et al., 2021) models latent editing as a continuous conditional flow: it learns a bijective mapping in latent space conditioned on attribute values (age, pose, gender, etc.), enabling smooth attribute-conditioned sampling and editing [1]. In StyleFlow, one can fix most aspects of the face and continuously change a specified attribute like gradually make the person older with high fidelity. Similarly, StyleCLIP leverages CLIP text embeddings to find latent directions: it either optimizes a latent to match a text prompt or trains a mapper network to translate text into a direction in W or S. This yields a flexible text-based interface for latent edits ("smile", "pikachu", "gothic") without requiring explicit labeled data. [31].

For image editing tasks, one often needs to map a real photo into StyleGAN2's latent space. GAN inversion methods seek a latent code (in W, W^+, or even feature space F) that reproduces the input under the generator. According to [24], GAN-Inversion methods suffer from fidelity-editability trade-off where optimizing in W^+ (high capacity) yields near-perfect reconstruction, but the resulting w^+ may lie outside the "safe" distribution, making edits unpredictable. Conversely, constraining to W yields more stable edits but higher reconstruction error. Modern techniques address this: encoder-based methods like pixel2Style2pixel (pSp) [32] or encoder4Editing (e4e) [37] quickly predict an approximate W^+ code; then post-processing (latent optimization or a "pivotal tuning") refines it. For example, pSp's encoder directly outputs 18 layer-specific style vectors into W^+, inverting a real face in one forward pass. Such hybrid inversion plus fine-tuning allows one to start with an interpretable latent in W/W^+ and then apply the semantic edits learned for synthetic images.

Ultimately, StyleGAN2's latent space serves as a controllable image model. One can generate images with specified attributes by sampling z and then steering $w = M(z)$ in desired directions [31]. For instance, adding a "smile" direction to w yields smiling faces. More generally, the latent allows fine-grained tuning: by selecting w^+ values

per layer, one can combine styles from different images or enforce multi-attribute conditions. StyleGAN2's latent space, especially W and S , provides a semantically meaningful coordinate system. Its evolution from earlier GAN latents (simple Z) to this multi-space structure underpins the state-of-the-art in GAN-based image editing and disentangled generation.

2.4 Attribute Editing in GAN Latent Spaces

Modern Generative Adversarial Networks (GANs) have shown that the latent codes controlling image generation often embed rich semantic structure. In particular, many high-level attributes (e.g. age, smile, pose) correspond to almost-linear directions in the latent space [17, 35]. Once a direction vector d for an attribute is found , editing is done by simple latent arithmetic $w_{edit} = w + \alpha d$ where w is the original latent code α is a scalar step size, and d is the learned attribute direction. For example, InterFaceGAN treats attributes as binary labels and trains a linear SVM in the latent space; the normal vector of the separating hyperplane becomes d [35]. Style-based GANs (StyleGAN1/2) enhance this by using a mapping network to produce an intermediate latent W space that is empirically more disentangled than the input Z space [22, 40].

There are two main approaches to finding attribute directions. Supervised methods use labeled data: for a given attribute one can train a linear classifier (e.g. on whether images have glasses) on a set of latent vectors. The classifier's weight vector (normal to the decision boundary) becomes the edit direction. InterFaceGAN demonstrated this works for many facial attributes [35]. There are two main approaches to finding attribute directions. Supervised methods use labeled data: for a given attribute one can train a linear classifier (e.g. on whether images have glasses) on a set of latent vectors. The classifier's weight vector (normal to the decision boundary) becomes the edit direction. InterFaceGAN demonstrated this works for many facial attributes. Unsupervised methods do not require labels: GANSpace, for instance, collects many random latents and applies Principal Component Analysis to find dominant directions. These principal axes often correspond to pose, lighting, or object type. Other factorization techniques, like SeFa, and style-space channel-searchers similarly find promising axes with little or no supervision [17, 34, 35].

Once a direction d is chosen, editing is simply walking along that direction in latent space. Concretely, after inverting or sampling a code w, one produces an edited image by generating from $w + \alpha d$ for various α. InterFaceGAN likewise shows that adding the gender vector shifts a face's apparent gender. In general , Attribute Editing Models use the same principle of finding d or Δw and setting $w_{edit} = w + \alpha d$ to change attributes in an image [17, 23, 35, 40]

Importantly, these methods rely on latent disentanglement: the idea that different factors are (at least partly) separable. StyleGAN's design helps here: its intermediate latent W is empirically more disentangled than the raw Z space [40]. StyleGAN2's per-channel style parameters ("StyleSpace") have been shown to control very localized attributes (e.g. hair color, background elements) with minimal interference. In contrast, earlier GANs had more entangled latents, making clean edits harder [11, 33].

However, disentanglement is not perfect. Most discovered directions still have residual cross-attribute effects. For example, one edit vector might change both "smile" and

"age" together. This entanglement is noted in the literature: many control directions affect multiple attributes and can be "non-local" [40]. To mitigate this, some methods perform conditional manipulation: e.g. subtracting out the projection of one direction on another to isolate each effect. But instability remains when editing multiple attributes at once, often producing unrealistic images if not handled carefully.

Attribute editing assumes that one has latent code w to edit . To apply editing to real photos, a critical step is GAN inversions. By finding a latent code that faithfully reproduces a given real image, one can apply learned semantic manipulations ,adjusting age, hairstyle, or pose, to that image [2, 4]. We find a latent code z^* whose generator output matches the target image. Once z^* is found , one can apply the w_{edit} formula to manipulate the image attributes. According to [41], there 2 gan inversion methods. Namely, the learning-based(encoder-based) and optimization based Gan inversions.

- Optimization-based inversion solves for z^* by backpropagation: it iteratively adjusts z to minimize reconstruction loss $||G(z) - x||$ This often yields high-fidelity reconstructions but can be slow and may get stuck in local minima.
- Learning(Encoder)-based trains a neural network $E(\cdot)$ to map images to latent codes. A trained encoder can invert images quickly, though it must be carefully regularized to align with the pretrained generator. Hybrid approaches use an encoder for an initial guess followed by optimization refinement [41].

GAN inversion is what allows editing of real images. For instance, the survey by [41] illustrates how an inverted code z^* can be altered by adding attribute vectors n_1, n_2 to change the age or smile of a real face. They note that after inversion, the same latent-space editing techniques apply "without requiring any ad-hoc supervision" However , inversion quality has limits: early models like Progressively Growing Gan (PGGAN)[20] were hard to invert faithfully, whereas StyleGAN's architecture yields much better inversions [22]. Imperfect inversion can blur details or introduce artifacts, which in turn limits how well edits carry over to the final image.

2.4.1 Attribute Group Editing

Beyond single images, recent work uses latent editing for few-shot image generation: synthesizing new images of a novel category given only a few examples. The idea is to treat each new class as a 'set of attributes' and apply the learned edit directions to produce more samples. A pioneering approach is Attribute Group Editing (AGE) [12]. AGE assumes that images are combinations of category-relevant and category-irrelevant attributes. It represents a category by its class embedding w_c: the mean latent of the few examples. The attributes that define the category (e.g., animal shape) are captured in w_c. AGE then learns a sparse dictionary of category-irrelevant attribute directions (e.g. fur color, pose) by doing sparse coding on the differences $w_{sample} - w_c$ for many samples. The key assumption is that an attribute direction (such as 'smiling' or 'red flower') is shared between categories. To generate new images of an unseen category, AGE starts with the class embedding w_c and adds combinations of these dictionary directions, creating diverse variations while keeping the features of the core category fixed. This editing-based generation requires no GAN retraining and can produce high-quality, diverse images even from one or few inputs.

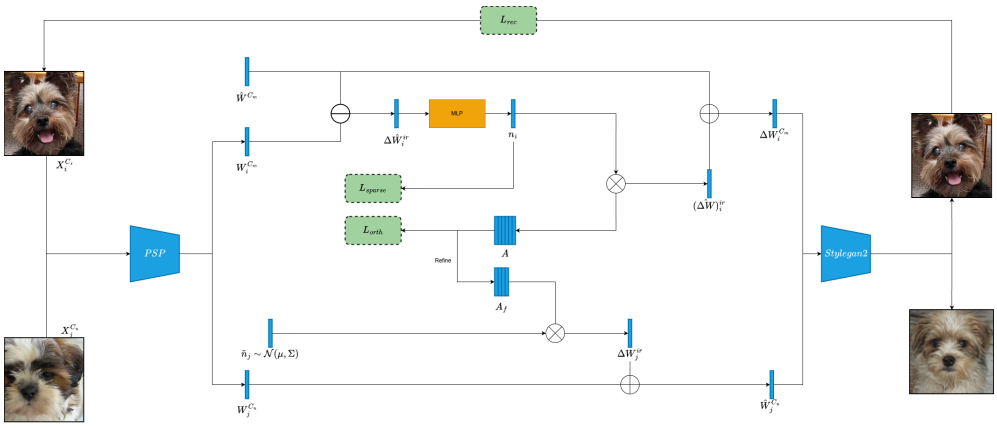

Fig. 2 *Attribute Group Editing Framework*

AGE provides a powerful framework, but has some limitations. The SAGE (Stable AGE) follow-up aims to improve class consistency. Instead of editing from just one example, SAGE uses all a few-shot images to robustly estimate the class embedding. It leverages the same attribute dictionaries but selects appropriate directions based on the new class's similarity to the seen classes. In effect, SAGE 'injects the whole distribution of the novel class into StyleGAN's latent space', preserving the identity of the category. This leads to more stable outputs that help downstream tasks (e.g., classification) perform better on the generated images [13].

Most recently, TAGE (Trustworthy AGE) has extended this line further. TAGE also uses a codebook (dictionary) of attributes, but introduces transformer-style modules. Its Codebook Learning Module (CLM) finds semantic directions for category-related and unrelated attributes and builds a sparse dictionary. A Code Prediction Module (CPM) then uses global combinatorial reasoning (long-range attention) to predict the best edit codes for a given few-shot class, improving diversity and accuracy. Crucially, TAGE adds a Prompt-Driven Semantic Module (PSM): it generates textual or semantic prompts that are injected into transformer layers, guiding fine-grained editing [44].

2.4.2 Stable Attribute Group Editing

Stable Attribute Group Editing (SAGE) was proposed as a method to overcome such limitations of AGE. Instead of editing from a single exemplar, SAGE initially employs all the given few-shot images to estimate a stable class-center embedding. It does so by capitalizing on the pretrained category-relevant attribute dictionary (learned from visible classes) to linearly represent the novel class centroid in the latent space. As described by Ding et al., since there are not a sufficient number of examples to calculate a credible mean directly, "the category-relevant dictionary... is complete enough to cover the unique attributes of any related categories," a linear combination of the dictionary atoms can "recover the class-specific features of the unseen classes" [13].

In practice, SAGE projects the few-shot samples onto such an attribute dictionary to separate their shared class features from each other, essentially removing

the attributes not related to categories and retaining the attributes that are related. According to the authors, such a disentanglement also helps the model to estimate a more accurate class embedding even for a very small number of samples. The experiments also indicate that the estimated embedding from only three face images well approximates the ground truth centroid over hundreds of actual instances.

In the same paper, the weights from the projection are utilized for adaptively choosing which category-irrelevant editing directions to use. Taking cues from biological taxonomy, the authors make the assumption that categories sharing the same relevant characteristics which is being members of the same genus, for example they also share compatible and safe irrelevant characteristics. Consequently, SAGE picks the category-irrelevant directions from the highly similar seen classes depending on the projection coefficients. The approach enables the method to introduce the distribution of the unseen class to StyleGAN's latent space by essentially repositioning the generated sample cloud towards the actual class manifold. Anchoring the synthesis about this reconstructed embedding and then carrying out filtered edits, SAGE manages to maintain class identity despite generating diverse and realistic variations.

This refinement of AGE yields clearer gains in category consistency and diversity. Quantitative results show that SAGE produces samples with significantly better quality (lower FID) and greater diversity (higher LPIPS) than AGE and competing few-shot methods. For example, without retraining the GAN, "SAGE generates images with higher quality and diversity" than prior fusion- or transformation-based approaches, and its class-embedding mapping makes it "much more stable than AGE". Qualitatively, Ding et al. illustrate that AGE could produce bizarre artifacts (a "cat with a dog nose" or a "bird with two heads") when editing from extreme inputs, whereas SAGE's tailored embedding and adaptive edits yield realistic variants without such errors. In the latent space of an unseen class, AGE's few-shot samples often lie at extreme positions and its generated points cluster narrowly around them. By contrast, SAGE's injections form a much wider latent cloud that more closely matches the true distribution. The authors summarize that SAGE "relocates the whole distribution of the novel class in the latent space, thus effectively avoiding category-specific editing and enhancing the stability". Overall, Ding et al. report that SAGE achieves "more stable and reliable few-shot image generation" with significantly better category retention than previous methods.

Nonetheless, SAGE does not solve all challenges. The authors note that its generative range is still more restricted than that of real data: the "dynamic range" of editable variations in SAGE is smaller than in the true class. In practice, SAGE currently uses a uniform editing intensity for all directions (since estimating per-attribute scales from few samples is infeasible) and its dictionary contains only attributes that recur across the seen classes. Consequently, some subtle class-specific variations may still be missing. As the paper observes, "SAGE simply assumes a unified intensity for editing direction sampling," and capturing attribute-dependent editing amplitudes or class-specific edits remains an open issue. Moreover, while classification performance is improved over AGE, it still generally lags behind a model trained on real examples. This gap – and the reliance on a pretrained StyleGAN inversion as the generator –

highlight future work: extending SAGE's ideas to handle multi-scale attributes and other generative models may be needed to fully close the gap for few-shot tasks.

2.4.3 Datasets and Applications

Attribute editing techniques are typically developed and evaluated on diverse image domains. The datasets Animal Faces HQ and Flowers are common: they have many labeled attributes and StyleGAN models are readily available for them. Few-shot generation research has also expanded to other domains. For instance, the Flowers dataset has 102 flower species [16], and Animal Faces (or AnimalFacesHQ) has 149 animal species.

Latent editing has practical use in data augmentation (few-shot generation) and interactive image manipulation. For example, one can start with a single animal or flower image and synthesize variants by editing color, pose, or environment using learned directions. However, note that many attributes do not directly map across domains; cross-domain transfer of semantic concepts remains challenging.

2.4.4 Attribute Editing limitations and challenges

Despite their promise, latent-space attribute editing methods face several challenges [12, 13, 24, 35, 40, 41]:

- Attribute entanglement. Discovered directions often change multiple features at once. For instance, moving along an "age" vector might also alter "expression". Entangled edits require careful design (e.g. subtracting projections) or more sophisticated disentanglement techniques. Fully disentangling a GAN's latent space is still an open problem.
- Multi-attribute interactions. Applying more than one edit vector can be unstable. Sequential edits may accumulate artifacts, and combining directions (e.g. "add glasses" and "increase smile") can produce unrealistic results if the attributes interact in complex ways. Ensuring consistent multi-attribute editing (for example, editing several facial features simultaneously without distortion) remains tricky.
- Inversion quality. Editing real images relies on GAN inversion. If the inversion does not perfectly capture the original, edits may fail or degrade fidelity. Early GANs (like PGGAN) struggled with inversion.While StyleGAN produces better inverted results, truly lossless inversion is hard. Any residual reconstruction error limits how cleanly attributes can be edited.
- Generalization to diverse datasets. Most latent-editing research has centered on well-studied domains (faces, common objects). Extending these methods to fine-grained categories like birds or flowers raises issues. The learned GAN (or its latent semantics) may not capture useful directions in these domains, especially if attribute labels are scarce. Additionally, attributes meaningful in one domain (e.g. "nose shape" in faces) do not translate to others. New methods must learn or adapt directions that make sense for non-face classes.

2.5 GAN Inversion

GAN inversion refers to finding a latent code in a pretrained GAN (here StyleGAN2) that reproduces a given real image when passed through the generator. In practice, one inverts an image x by solving for a style code w such that the StyleGAN2 generator G outputs an image $G(w) \approx x$ [37]. This can be done via optimization (iteratively adjusting w for each image, which is slow) or via a learned encoder (a neural network that predicts w in one forward pass) [2, 37]. StyleGAN2 uses a mapping network from a base Gaussian z into an intermediate latent space W, which is more disentangled. Many inversion methods actually use an extended latent space W^+ (one 512-D style vector per layer) for greater expressiveness [4, 32, 37]. Once an image is inverted to a latent code, one can edit the image by manipulating that code (e.g. adding learned attribute directions or swapping style layers) and generating the edited image via G. High-quality inversion thus requires balancing reconstruction fidelity (low distortion between x and $G(w)$) with with editability (the code lies in a region where semantic edits are meaningful).

How GAN inversion works [2, 32, 37]:

- Pretrained StyleGAN2: Use a generator G trained on a target domain. Fix its weights.
- Latent spaces: Identify the latent space to invert into (typically W or extended W^+ sometimes an additional feature space F).
- Encoder or optimization: For encoder-based inversion, train a feed-forward network E to map images to latents. For optimization-based inversion, for each image x optimize W by minimizing a loss $\|x - G(w)\|$. Encoder methods are much faster (one pass) but historically had worse fidelity than optimization.
- Architecture of encoders: Typically, an encoder is a convolutional backbone (often ResNet) that produces latent parameters. For example, pixel2style2pixel (pSp) uses a ResNet + feature pyramid, producing style vectors via map2style blocks for each scale. Encoder4Editing (e4e) is similar but outputs one base style plus per-layer offsets. ReStyle applies such encoders iteratively, and StyleFeatureEditor uses a two-branch encoder for both W^+ and feature-latents.
- Loss functions: Encoders are trained to minimize a combination of losses on reconstruction. Common losses are pixel-wise L2, perceptual (LPIPS) loss, and often an identity or feature loss to preserve semantics (e.g. using ArcFace or self-supervised features).Many methods also add regularization: for example, e4e adds L2 penalties on offsets to keep the final code near W, and some use adversarial (latent discriminator) loss to keep codes within the true latent distribution.
- Inference and editing: At test time, given image x, the encoder predicts a latent code (or code+offsets or code+features). This code is fed into the fixed generator G to reconstruct x. For editing, one modifies the latent (e.g. by style mixing or adding semantic direction vectors) and uses G to synthesize the edited image.

2.5.1 pixel2style2pixel (pSp)

pSp is an early StyleGAN2 encoder (CVPR 2021) that maps an image to the extended latent space W^+. It uses a ResNet-50 backbone with a feature pyramid producing

three scales of feature maps. Each scale is fed through small map2style networks to generate 512-D style vectors corresponding to StyleGAN's layers. The output is a full W^+ The encoder is trained with a weighted sum of losses: pixel L2, LPIPS perceptual loss, and an identity loss (ArcFace for faces, or a self-supervised ResNet for other domains). A small regularization term on the styles can also be used. By design, pSp prioritizes reconstruction fidelity. It "directly maps a real image into the W^+ latent space with no optimization required". In experiments it achieves low LPIPS and MSE reconstruction error. Because it predicts full W^+, pSp can accurately capture fine details of the image. After encoding, image editing is done by standard StyleGAN manipulations (e.g. adding latent offsets or style-mixing). For example, pSp naturally supports style-mixing for multi-modal synthesis: one can replace selected style vectors with those from a random code to generate variations.

pSp is fast (single forward pass) and yields high-quality reconstructions. It has been shown to preserve identity and details (e.g. hairstyle, lighting) better than many earlier methods. It also generalizes to various image-to-image tasks without retraining (e.g. frontalization, super-resolution) because of the rich StyleGAN latent space.

However , since pSp uses full W^+, the encoded latents can lie far from the original distribution learned in W. This can degrade "editability": edits in regions of W^+ far from the data manifold may produce unrealistic results. pSp's very high reconstruction fidelity sometimes comes at the cost of less coherent edits, compared to encoding into W or constrained spaces. [32]

2.5.2 Encoder4Editing (e4e)

Encoder4Editing (e4e, ICCV 2021) extends pSp by explicitly addressing the reconstruction editability trade-off. The key idea is to keep the inverted latent closer to the original W distribution even while allowing high-fidelity reconstruction. Architecturally, e4e still uses a ResNet+feature pyramid encoder, but instead of predicting all style vectors independently, it predicts one base style code w_0 and a small offset vector o_i or each of StyleGAN's 18 layers. The final style for layer i is $w_0 + o_i$.During training, two principles are enforced:

1. Progressive training: initially predict only w_0 then gradually allow one layer's offset at a time, progressing from coarse to fine layers.
2. Offset regularization: penalize the magnitude of the offsets (via L2) to keep $w_0 + o_i$ close to w_0 and thus close to the learned W manifold. A small latent discriminator (GAN loss in latent space) is also trained to distinguish real W samples from the encoder's outputs.

During training e4e uses the same image-space losses as pSp (L2, LPIPS, identity) and adds the above editability losses. The result is an encoder that strikes a balance: it produces reconstructions nearly as good as pSp, but the codes are "closer to W" so that semantic edits transfer more reliably. e4e still outputs an extended style code, but with built-in bias towards W. In practice one can edit e4e codes using the same methods (adding latent directions, style mixing), and edits tend to be more robust. For faces, e4e also incorporates a cosine identity loss (ArcFace or general ResNet) to preserve identity under reconstruction.

e4e greatly improves editability. It was shown to maintain coherent attribute manipulations (age, expression, pose) on real images better than pSp, with only a minor increase in reconstruction error. ts inference speed is still one forward pass (plus negligible overhead from the latent discriminator at training time). It generalizes to multiple domains (faces, cars, etc.) by using a domain-agnostic feature extractor for identity.

To achieve editability, e4e accepts a small drop in reconstruction fidelity. Very fine details may be slightly lost compared to pSp. Also, the progressive training and latent GAN add complexity, and tuning the trade-off weights requires care. The notion of "proximity to W " is somewhat loosely defined , so extremely out-of-distribution images (e.g. non-human faces) may still not be perfectly encoded. [37]

2.5.3 ReStyle

ReStyle (ICCV 2021) focuses on improving inversion accuracy by iterative refinement. Instead of a single encoder pass, ReStyle's encoder is applied multiple times in a feedback loop. Starting from an initial latent (the average w vector), ReStyle repeatedly concatenates the original image with the generator's current reconstruction and feeds them into the encoder. At each iteration t, the encoder predicts a residual $\Delta w^{(t)}$ to add to the current latent.

$$\text{At step t,} w^{(t)} = w^{(t-1)} + E(x, G(w^{(t-1)}))$$

,

with $w^{(0)}$, set to the average latent, and $G(w)$, the current output image. The encoder is trained to minimize reconstruction loss at each step (summing L2+LPIPS over all iterations). ReStyle can be applied on top of any base encoder (e.g. pSp or e4e). In implementation, the authors actually simplify those encoders by using only the final feature map (no pyramid). Each iteration uses a fresh forward pass through the same network (weights are shared across steps). The training, therefore, unrolls a small multi-step network with losses on each reconstructed image.

ReStyle yields a more accurate latent code after its iterations, meaning $G(w)$ more closely matches the input. This generally improves any downstream editing (since the base reconstruction is better). In other words, it aims to match the quality of per-image optimization (but much faster). The final code is in W^+ and can be edited normally.

ReStyle significantly improves reconstruction quality compared to standard feed-forward encoders. It achieves near-optimization accuracy with only a small inference-time cost (typically 3-5x forward passes). Because it uses the same encoder architecture at each step, it avoids complex modifications. It can even be viewed as a learned optimization in latent space.

The iterative loop means inference is slower (several passes). Although small (e.g. 5 steps), it is no longer a single-shot encoder. Also, ReStyle does not explicitly enforce editability constraints - it focuses on fidelity. It inherits the latent space choice of the base encoder: if initialized in W^+ the result can still lie outside the original W distribution. Finally, training is more complex (unrolling multiple steps with deep supervision) and may be sensitive to number of iterations. [2]

2.5.4 StyleFeatureEditor

StyleFeatureEditor (SFE) introduces a new latent representation combining StyleGAN's usual style codes, $worw_i$, with a feature tensor F_k from an intermediate generator layer. Intuitively, the feature tensor F_k , high-dimensional spatial features , captures fine detail that W^+ alone may miss. The method has two networks: an inverter I and a feature editor H. The inverter I takes an image X and predicts both a style code $w \in W^+$ and a feature map F_k. Specifically , I first maps X to w via linear layers, and also to an intermediate feature F_{pred} . Then it fuses F_{pred} with the actual generator's feature. to produce the final inversion feature F_k.

Crucially, StyleFeatureEditor trains a separate editor network $H(F, \Delta)$ that adjusts the feature tensor for editing. Given an editing direction d in n latent space, one computes the change $\Delta = F_k(w + d) - F_k(w)$ by sampling from a pretrained encoder. Then H is trained to take the inverted feature F_k and Δ and output the edited feature $F'_k = H(F_k, \Delta)$. In inference, for a desired edit d, one computes $w' = w + d$, uses H to updated $F \Rightarrow F'_k$ and generates the edited image from w', F'_k.

StyleFeatureEditor undergoes 2 phase training:

1. Inverter I is trained on real images with standard losses (L2+LPIPS+identity, plus latent regularization and adversarial loss).
2. Editor H is trained on synthetic pairs: for each real X, a pretrained W^+ encoder produces w, an edit d is sampled, yielding, X'_E by adjusting H(inverter frozen).

StyleFeatureEditor achieves extremely high reconstruction quality and preserves detail even under editing. In experiments it produces nearly indistinguishable inverted images, and crucially it "preserves most" of those fine details when editing. By operating directly on feature maps, it can maintain textures and fine structure that pure latent edits often lose. It is capable of editing challenging out-of-domain examples and does not rely solely on W or W^+ for content. [4]

2.6 Few-Shot Image Generation in GANs

Few-shot image generation (FSIG) extends few-shot learning to generative modeling: the goal is to synthesize novel images for a new category given only a handful of training examples. This contrasts with few-shot classification, where techniques like transfer learning have proven effective [8]. In FSIG, however, naively fine-tuning a powerful generative model on few examples leads to overfitting and mode collapse. According to [45], modern GANs "tend to replicate the training samples" when adapted to only 10 target images. As a result, FSIG methods typically transfer rich prior knowledge from a large source domain and then adapt carefully to the new domain. This is often framed as a style-transfer or domain-adaptation problem: a generator pretrained on a large dataset ,like FFHQ faces, is fine-tuned on a small target set without access to the source data [6, 45]. The key challenge is preserving the source model's diversity and high-fidelity output while learning the new style from very limited data.

Generative Adversarial Networks (GANs) have been the dominant framework for FSIG. The standard paradigm is to take a GAN (often StyleGAN or BigGAN) pretrained on a large source dataset and adapt it to the new class. Early approaches

simply fine-tuned all weights on the few-shot target (TGAN) [45], but this quickly overfits (the generator memorizes the few examples). Numerous strategies have been proposed to prevent collapse and preserve diversity during adaptation. or instance, FreezeD freezes some high-resolution layers of the discriminator, ADA (adaptive data augmentation) introduces random augmentations during training, and EWC applies an Elastic Weight Consolidation penalty on important weights [28]. Another approach Cross-domain Correspondence (CDC) preserves the relative distances between different source-domain samples in the target embedding space [30]. In practice, Zhao et al. (2022) found that most of these GAN-adaptation methods converge to similar image quality after many iterations – the real difference is how slowly they lose diversity during training [45]. In other words, diversity degradation (mode collapse) is the main bottleneck: methods that slow down this collapse (e.g. by freezing parameters or using strong regularization) tend to perform better overall.

Building on this insight, Zhao et al. propose a contrastive learning scheme to retain diversity. Their Dual Contrastive Learning (DCL) maximizes mutual information between the source and target generators: the idea is that the same latent input z should map to semantically corresponding images before and after adaptation. Concretely, they use a contrastive loss on multi-level features of the generator and discriminator to enforce that source and target outputs remain similar in high-level content. This preserves the source generator's rich variation in the adapted model, yielding state-of-the-art results on several datasets [45]. Similarly, Hong et al. (2020) introduce DeltaGAN, which decomposes generation into a reconstruction sub-network (that computes an intra-class "delta" transformation between two samples of the same category) and a generation sub-network (that applies a sample-specific delta to a base image) [19]. DeltaGAN's adversarial "delta matching" encourages high diversity by learning many distinct transformations from the limited examples.

GAN-based FSIG methods share a transfer-learning theme: leverage a pretrained model's knowledge and carefully regularize the adaptation. Typical training strategies include fine-tuning, parameter freezing, data augmentation, and contrastive or distance-preserving losses. Their success is usually measured by metrics like Frechet Inception Distance (FID) or LPIPS (perceptual diversity).

Transfer learning is a commonly used training strategy in generative models that starts from a pre-trained model (GAN, VAE, or diffusion) and updates it using a few target images. Pure fine-tuning can often overfit the model, making it difficult to generalize well [45]. Meta-learning involves training on simulated few-shot tasks so that it can adapt quickly. Contrastive/Distance Losses are used to enforce that the adapted generator preserves relationships between different domains. Data augmentation and synthesis create pseudo-examples by replacing source images with those in the target fine-tuning process. While some GANs/VAEs may use MAML or Reptile, most often they still need per-task fine-tuning and struggle to generate sharp images [19]. In contrast, meta-learned GANs/VAEs have been attempted but are usually limited by their overfitting problem [19]. Meta-Learning involves training on many simulated few-shot tasks so that it can adapt quickly. Data augmentation and synthesis create pseudo-examples by replacing source images with those in the target fine-tuning process. In

this way, humans can better understand how to generate realistic images based on the data they have available.

Evaluation strategies are also noteworthy. FSIG is evaluated on cross-domain tasks such as generating artistic or cartoon versions of FFHQ faces, or adapting LSUN Church to "Haunted House" style, or similar sketch-to-photo tasks. Metrics include FID for quality (when enough data exists) and LPIPS or recall for diversity. [45] emphasize that with only 10 real images, FID is unstable, so they instead use intra-LPIPS (average pairwise LPIPS of generated set) to quantify diversity .

Despite these advances, few-shot image generation remains challenging. Surveys note that learning from very few or zero examples is still "a serious challenge" [36]. Key open issues include: catastrophic forgetting (the model loses the source distribution entirely), mode collapse (outputs lack diversity), and evaluation (how to measure quality/diversity with tiny real sample sets). Overfitting to the few examples is ubiquitous: even with strong regularization, generators tend to memorize salient features. Furthermore, different generative families have trade-offs – GANs are fast and high-quality but brittle, VAEs are stable but blurrier, and diffusion models are powerful but computationally heavy. A brief comparison: GANs excel at sharp images but require careful balancing of losses; VAEs naturally encourage covering the whole distribution (which aids diversity) at the cost of some sharpness; diffusion models sample iteratively and often capture fine detail, but naively adapting them can be even more parameter-sensitive than GANs [6].

3 Methodology

3.1 Research Design

This paper, "FeatureSAGE: Stable Attribute Group Editing With Integrated StyleFeatureEditor Encoder for Enhanced Few-Shot Image Generation," adopts a quantitative approach with an experimental research design to evaluate improvements to the existing SAGE framework. In particular, the study integrates the GAN inversion module of SAGE with StyleFeatureEditor (SFE), incorporating its Feature Editor module to enhance latent inversion and attribute editing functionality under a one-shot setting, where attribute manipulation is conditioned on a single reference image per test instance. The research employs a quantitative methodology using established automated metrics for objective evaluation. Reconstruction quality is assessed using Fréchet Inception Distance (FID) and Learned Perceptual Image Patch Similarity (LPIPS).

3.1.1 Experimental Setup

The experimental setup is designed to evaluate the effectiveness of the proposed FeatureSAGE framework in comparison to the existing SAGE method. The evaluation follows a quantitative, experimental research design that involves controlled changes to a single independent variable and the measurement of multiple dependent variables using standardized metrics.

- **Independent Variable**

  - GAN Inversion Method: The primary manipulation involves substituting the original Pixel2Style2Pixel (pSp) encoder used in SAGE with the proposed StyleFeatureEditor (SFE) encoder, which includes an integrated feature editing module for enhanced semantic attribute control.

- **Dependent Variable** To objectively evaluate the effectiveness of the GAN inversion and editing pipeline, the following metrics are employed:
  - Reconstruction Quality
    1. Fréchet Inception Distance (FID): Measures distribution-level similarity between generated and real images.
    2. Learned Perceptual Image Patch Similarity (LPIPS): Measures perceptual similarity between original and reconstructed images based on deep features.

- **Controlled Variables**
  1. Generator Architecture: A fixed, pretrained StyleGAN2 generator is used in both configurations for consistent image synthesis capabilities.
  2. Training Dataset: Both configurations use the same datasets, FUNIT Animal Faces and 102 Flowers, with the same preprocessing and data-splitting.
  3. Evaluation Protocol: The same unseen categories from the train-test split data are used for one-shot testing in both configurations. Evaluation is conducted using identical random seeds, sample counts, and query sets.
  4. Loss Functions: Reconstruction and editing losses are applied consistently across both systems.
  5. Optimization Settings: Both models utilize the same optimizer type, learning rate, batch size, and training epochs.
  6. Evaluation Metrics: All quantitative metrics, FID and LPIPS, are computed using the same implementation and parameters for consistency.
  7. Hardware and Software Environment: All experiments are conducted using the same hardware and PyTorch environment to avoid performance inconsistencies. The system specifications are reported in Section 3.2.

- **Experimental Conditions**
  Two experimental configurations are used:
  - SAGE (Baseline): Employs the original pSp encoder with StyleGAN2.
  - FeatureSAGE (Proposed): Integrates StyleFeatureEditor (SFE) encoder and a Feature Editor.

3.1.2 Few-Shot Experimental Setting

In this study, the term *few-shot* refers to the number of reference images available per target class or identity used to construct class-level latent representations for attribute group editing and downstream evaluation. A one-shot setting utilizes a single reference image per target class, representing an extreme data-scarce scenario in which attribute editing and inversion must be performed with minimal visual information.

This one-shot configuration is applied consistently across all datasets and experimental conditions for both the baseline SAGE and the proposed FeatureSAGE framework. By explicitly controlling the evaluation to a single reference image per class, the experimental protocol enables a systematic assessment of FeatureSAGE's robustness, stability, and generalization capability under severe data limitations. The few-shot constraint is applied during test-time adaptation and evaluation on unseen classes, while the StyleGAN2 generator and attribute directions are learned from large-scale seen-category data.

3.2 Hardware and System Requirements

All experiments were conducted on a virtual machine instance with GPU pass-through hosted on the Caraga State University ICTC supercomputer server. The same hardware and software environment was used for the baseline and proposed configurations so that differences in FID and LPIPS could be attributed to the inversion and editing pipeline rather than to system-level variation.

- **CPU:** QEMU Virtual CPU version 2.5+ with two processors operating at 2.19 GHz.
- **RAM:** 32.0 GB system memory.
- **GPU:** NVIDIA GRID V100D-32A, a virtualized Tesla V100 GPU with 16 GB HBM2 memory.
- **Storage:** 1 TB server HDD storage. The environment stores the FUNIT Animal Faces and 102 Flowers datasets, pretrained StyleGAN2 generator checkpoints, pSp and StyleFeatureEditor checkpoints, generated images, and evaluation outputs.
- **Software environment:** The implementation used Linux Ubuntu 24.04 LTS with a PyTorch environment, a pretrained StyleGAN2 generator, the pSp encoder baseline, the StyleFeatureEditor Inverter and Feature Editor, MTCNN-based preprocessing for Animal Faces, Adam optimization, mixed-precision training, Inception-v3 features for FID, and deep network features for LPIPS.

The 16 GB V100-class GPU is particularly important because FeatureSAGE performs multiple generator and inversion passes and injects intermediate feature tensors during synthesis. Mixed precision was enabled during Feature Editor training to reduce memory usage while preserving the same experimental protocol across comparisons.

3.3 Data Preprocessing

3.3.1 Data Sources

This research draws from a range of publicly available datasets with a varied scope of visuals such as animal faces and flowers. The datasets are popular for generating and attribute-editing applications because of their quality and well-labeled categories. For the purpose of this research's emphasis on few-shot image synthesis, only a minority of the images from each dataset are employed. Specifically, a minimal set of representative samples per class are sampled to replicate the few-shot learning situation. The datasets used are:

1. **FUNIT Animal Faces Dataset**

Fig. 3 Sample images from the FUNIT Animal Faces dataset. Source: [25]

Figure 3 shows representative samples from the FUNIT Animal Faces dataset. The dataset is comprised of 117,484 512×512 resolution high-quality animal face images and is organized in 149 classes, where each class is one animal species. Each domain includes a diverse range of species and breeds, ensuring variation in appearance while maintaining consistent facial alignment to support generative model training. The images were curated from openly available sources and standardized in resolution and alignment, with the eyes centrally positioned to provide consistency for learning. A subset of the images is reserved as a test set, with the remainder used for training. The diversity and quality of the FUNIT Animal Faces dataset make it a strong benchmark for generative and attribute-editing models in few-shot settings [25].

2. **102 Flower (Flowers)**

Fig. 4 Sample images from 102 Flower Dataset. Source: [29]

Figure 4 shows sample images from the 102 Flower dataset, which comprises 102 flower classes present in the United Kingdom. There are between 40 and 258 images in each class. The flowers covered by the dataset are regularly occurring species in the UK. The dataset shows a great deal of variation in the sizes, pose, and lighting conditions of the flowers, which is an added complexity for the task of classification. Moreover, there are large intra-class differences for some categories, and there are many categories that are visually extremely similar to each other. All such attributes make the dataset extremely suitable for fine-grained visual categorization. The dataset was also used for analysis and visualization using isomap dimensionality reduction based on shape and color descriptors, which revealed the structure of the different flower categories. As for the sizes of the images, the raw images in the dataset are of varying sizes. For instance, one raw image from the dataset measures 500×667 pixels [29].

3.3.2 Data Preprocessing Steps

For all datasets, the following preprocessing steps are applied:

- **Resizing and Normalization:** Images are resized according to the generator and evaluation setting, including the reported 1024×1024 experiments where applicable, and pixel values are normalized to the range $[-1, 1]$ for input to StyleGAN2.
- **Filtering:** Extremely low-quality or mislabeled images are removed.
  - For FUNIT Animal Faces, MTCNN (Multi-Task Cascaded Convolutional Networks) is used to crop and horizontally align faces so the eyes are leveled.
- **Train-Test Splitting:** The approach is assessed on few-shot image domains by partitioning each dataset into seen classes for model fitting and unseen classes for testing.
  - **Animal Faces.** Out of the total categories, 119 are used as seen classes for training, while the remaining 30 serve as unseen classes for evaluation.
  - **Flowers.** The study assigns 85 categories to the training, or seen, split and reserves 17 categories for the test, or unseen, split.
- **Augmentation:** No data augmentation is used beyond centering and flipping, since the evaluation focuses on the given domains. All datasets and preprocessing pipelines are documented for reproducibility.

3.4 Model Architecture

This study proposes a modified SAGE framework named **FeatureSAGE**, which integrates several modules to enhance few-shot image generation. The architecture combines a pretrained StyleGAN2 backbone, a StyleFeatureEditor Inverter and Feature Editor, and SAGE's attribute group editing mechanism. The design aims to enable fine-grained, disentangled, and class-consistent image synthesis from limited input data. The main modules are as follows:

1. **StyleGAN2 Generator:** As in SAGE, the synthesis module of a pretrained StyleGAN2 generator is used to generate images from inverted latent codes. In FeatureSAGE, the generator also receives edited intermediate feature tensors from the Feature Editor so that the output image is conditioned by both latent-space edits and feature-level details.
2. **StyleFeatureEditor Inverter and Feature Editor:** StyleFeatureEditor (SFE) by [4] enables editing in both W^+ latents and F-latents. This allows the model to reconstruct finer image details and preserve them during editing. In FeatureSAGE, SFE is integrated into the GAN inversion module $I(\cdot)$ of the original SAGE framework to improve latent inversion and attribute editing. According to [13], reconstruction with pSp, ReStyle, and HyperStyle may lose local details, reducing the accuracy of reconstructed images by more than 5% compared with standard data. This motivates the use of the StyleFeatureEditor Inverter $I(\cdot)$ and Feature Editor $H(\cdot)$ as the inversion and feature-editing components of the proposed framework.

3.4.1 Feature Extraction and Latent Representation

FeatureSAGE extracts three types of representations before image synthesis. First, each input image is encoded into a W^+ latent code. The W^+ space contains layer-specific StyleGAN2 style vectors; in this study, it is represented as 18 layers with 512-dimensional style information per layer. This representation is important because it supports semantic attribute editing while preserving the layer-wise structure used by StyleGAN2.

Second, the StyleFeatureEditor Inverter predicts an intermediate feature tensor F_k, where k denotes the selected StyleGAN2 generator layer used for feature injection. In the decoder block of this study, the feature tensor at layer 9 is used. The feature tensor preserves local visual details that may be lost when editing only in W^+, such as texture, color boundaries, and fine structural information.

Third, the framework computes class-level and attribute-level latent representations. For a category C with N_C available latent codes, the class embedding is the mean latent representation:

$$e^C = \frac{1}{N_C} \sum_{i=1}^{N_C} w_i^C. \tag{1}$$

Equation (1) represents the category-relevant component because it summarizes the shared structure of a class. The residual between an image latent and its class embedding is then used to represent category-irrelevant variation, such as pose, color, texture, orientation, and background. These residuals are the basis for attribute factorization and adaptive editing.

The training and test sets are denoted as

$$D_{\text{train}} = \{x_i^{C_s}\}_{i=1}^{S}, \qquad D_{\text{test}} = \{x_i^{C_u}\}_{i=1}^{U}, \tag{2}$$

where C_s denotes seen categories, C_u denotes unseen categories, S is the number of seen-category samples used in training, and U is the number of unseen-category samples used for testing. Equation (2) defines the data split used by the one-shot evaluation protocol.

3.4.2 Framework Flow

Figure 5 shows the complete FeatureSAGE framework. The pipeline follows a sequential input-processing-output structure:

1. **Input:** A one-shot reference image from an unseen category is preprocessed through resizing, alignment when applicable, and normalization before inversion.
2. **Latent and feature extraction:** The image is encoded into W^+ latent codes and intermediate feature tensors. The pSp encoder is used in the baseline path, while the proposed path uses the StyleFeatureEditor Inverter.
3. **Attribute factorization:** Seen-category latent residuals are decomposed into category-relevant and category-irrelevant components. The category-irrelevant directions are stored in dictionary A.
4. **Class embedding mapping:** The class-level latent representation is filtered to estimate the category-relevant embedding of unseen categories.

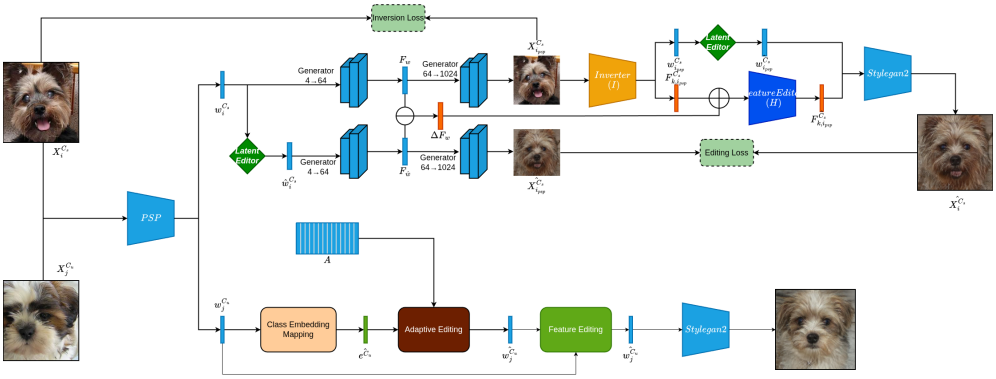

Fig. 5 FeatureSAGE framework

5. **Adaptive editing:** A category-specific subset of attribute directions is selected and sampled to produce stable edits for the unseen category.
6. **Decoder and output synthesis:** The edited W^+ latent and the edited intermediate feature tensor are passed to the StyleGAN2 generator. The output is a generated image that preserves class-relevant identity while modifying category-irrelevant attributes.

For the baseline training pair, a pretrained pixel2style2pixel encoder pSp takes a seen image $X_i^{C_s}$ and predicts a W^+ latent code:

$$w_i^{C_s} = \mathrm{pSp}(X_i^{C_s}), \qquad w_i^{C_s} \in W^+. \tag{3}$$

Equation (3) defines the pSp inversion step used to obtain the baseline latent code. In FeatureSAGE, the StyleFeatureEditor Inverter predicts both the latent code and intermediate feature tensor:

$$I(X_i^{C_s}) = \left(w_{i,\mathrm{sfe}}^{C_s}, F_{k,i}^{C_s}\right). \tag{4}$$

Equation (4) is used because SFE explicitly represents both global style information through $w_{i,\mathrm{sfe}}^{C_s}$ and local detail through $F_{k,i}^{C_s}$.

Using the latent editor, random directions are sampled from the sparse dictionary A to obtain an edited latent code $\hat{w}_i^{C_s}$. The original and edited latent codes are then synthesized by the StyleGAN2 generator:

$$(x_{i,\mathrm{psp}}^{C_s}, F_w) = G(w_i^{C_s}), \qquad (\hat{x}_{i,\mathrm{psp}}^{C_s}, F_{\hat{w}}) = G(\hat{w}_i^{C_s}). \tag{5}$$

Equation (5) provides the editing pair used in Feature Editor training. The images $x_{i,\mathrm{psp}}^{C_s}$ and $\hat{x}_{i,\mathrm{psp}}^{C_s}$ supply the reconstruction and edited targets, while F_w and $F_{\hat{w}}$ provide the feature-space difference used by the decoder.

3.4.3 Attribute Factorization

To enable disentangled editing, FeatureSAGE retains the SAGE attribute factorization stage. This stage learns category-irrelevant attributes in the latent space by modeling the difference between each sample latent code and the mean latent code of its category.

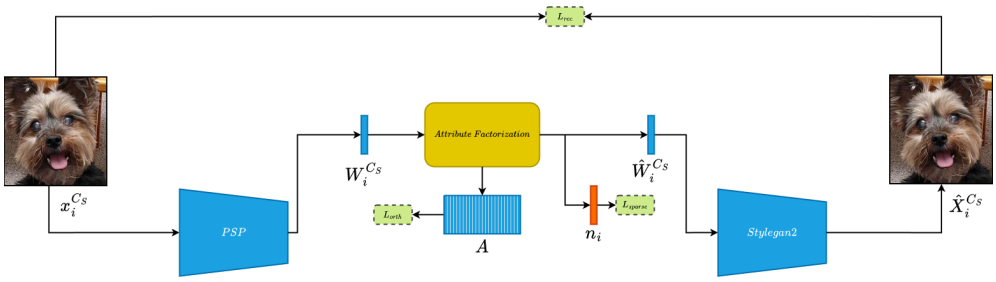

Fig. 6 Attribute Factorization Training Framework [13]

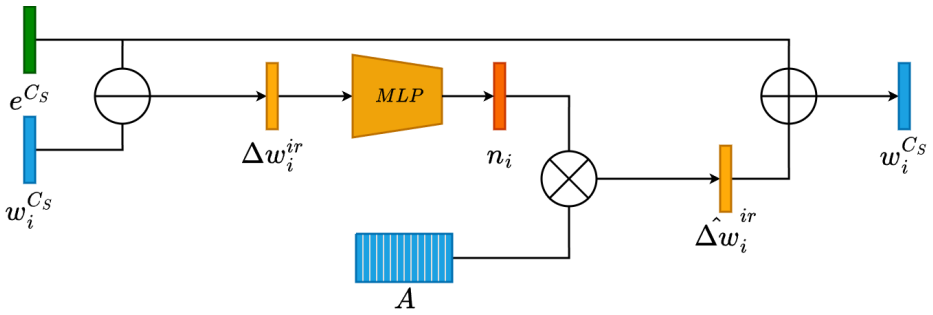

Fig. 7 Attribute Factorization Block, derived from [13]

In the Attribute Factorization block shown in Figure 7, the residual latent component is computed as

$$\Delta w_i^{ir} = w_i^{C_s} - e^{C_s}. \tag{6}$$

Equation (6) subtracts the category embedding from the image latent code, leaving variation that is not essential to the category identity. The residual is then approximated through dictionary directions and sparse coefficients:

$$\widehat{\Delta w_i^{ir}} = A n_i. \tag{7}$$

In Equation (7), A is the global dictionary of category-irrelevant attribute directions and n_i is the sparse code that selects which directions are active for sample i. The edited residual is added back to the class embedding to obtain an edited latent code.

Category-irrelevant attributes are features that influence how an object appears but are not essential to its identity or class membership. For example, a cat remains a cat even if its color, orientation, background, texture, or expression changes. Following [13], category-irrelevant editing is defined as

$$\begin{aligned} \text{edit}_{ir}(G(W_i^{C_s})) &= G\left(W_i^{C_s} + \alpha \Delta W^{ir}, \right. \\ &\qquad \left. H(F_{k,i}, \Delta W^{ir})\right) \\ &= \hat{X}_i^{C_s}. \end{aligned} \tag{8}$$

Equation (8) expresses the main editing operation. The scalar α controls edit strength, ΔW^{ir} is the category-irrelevant direction, $H(\cdot)$ edits the feature tensor, and $\hat{X}_i^{C_s}$ is the edited output image.

Given a latent code $W_i^{C_s}$ and its class embedding e^{C_s}, the category-irrelevant direction sample is obtained by

$$\Delta W_i^{ir} = W_i^{C_s} - e^{C_s}. \tag{9}$$

Equation (9) is the same residual principle as Equation (6), written for the W^+ latent tensor.

According to [13], a global dictionary $A \in \mathbb{R}^{18\times 512\times l}$ and a sparse representation $n_i \in \mathbb{R}^{18\times l}$ are learned from ΔW_i^{ir} using sparse dictionary learning:

$$\min_{n_i} \|n_i\|_0 \quad \text{s.t.} \quad \Delta W_i^{ir} = A n_i. \tag{10}$$

Equation (10) uses the L_0 constraint to encourage each sample to activate only a small number of dictionary directions, making each direction more semantically meaningful.

In practice, the sparse code is estimated through a multilayer perceptron:

$$n_i = \text{MLP}(\Delta W_i^{ir}). \tag{11}$$

Equation (11) makes the sparse dictionary learning step differentiable and trainable within the attribute factorization module.

3.4.4 Class Embedding Mapping

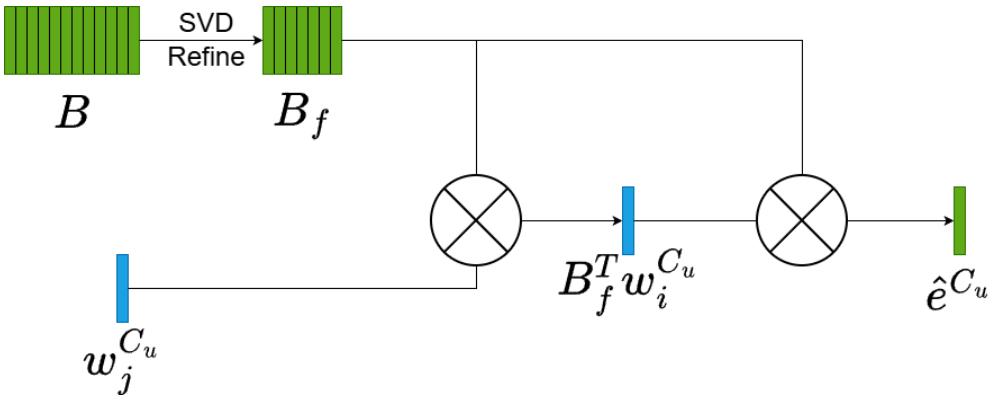

Fig. 8 Class Embedding Block, adapted from [13]

Compared with Attribute Group Editing (AGE) by [12], which randomly samples editing directions from A, SAGE uses class embedding mapping. The class embedding mapping block in Figure 8 identifies the latent directions that have the strongest effect on the target attribute group and uses them to guide later edits. This part of the framework remains unchanged from SAGE.

Let B denote the set of class embeddings from S seen categories. Since these embeddings are based on average latent codes, they may still contain minor interference from category-irrelevant attributes. Singular value decomposition (SVD) is applied to identify the most informative directions:

$$B = U_B \Sigma_B V_B^*. \tag{12}$$

In Equation (12), U_B contains the left singular vectors, Σ_B contains the singular values in descending order, and V_B^* is the conjugate transpose of the right singular vector matrix. The top-t_B directions in U_B are selected as the final category-relevant attribute dictionary $B_f \in \mathbb{R}^{18\times512\times t_B}$.

For an unseen category C_u, the class embedding is estimated by projecting its latent codes onto B_f and averaging:

$$\hat{e}^{C_u} = \frac{1}{N_u} \sum_{i=1}^{N_u} B_f B_f^T W_i^{C_u}. \tag{13}$$

Equation (13) defines $\hat{e}^{C_u}$ as the category-relevant embedding of unseen category C_u, where N_u is the number of available unseen-category latent codes and $\{W_i^{C_u}\}_{i=1}^{N_u}$ are the corresponding latent representations.

3.4.5 Adaptive Editing

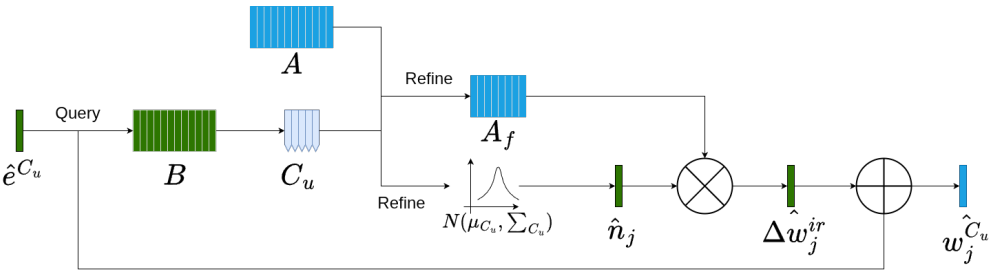

Fig. 9 Adaptive Editing Block, from [13]

Figure 9 shows the adaptive editing block of FeatureSAGE. Category-irrelevant qualities often have similar distributions in closely related categories but differ across distant categories. Therefore, adaptive editing selects the top-t_C seen categories whose class embeddings are closest to the unseen category embedding. To find the category-irrelevant editing directions that best fit C_u, the residual direction is back-projected into sparse-code space:

$$\hat{n}_i = A^{\dagger} \Delta W_i^{ir}. \tag{14}$$

In Equation (14), $A^{\dagger}$ denotes the pseudo-inverse of A, and $\hat{n}_i$ estimates how strongly each dictionary direction contributes to sample i.

The mean absolute activation across the selected similar seen categories is then computed as

$$\overline{|n|}^{C_u} = \frac{1}{t_C} \sum_{t=1}^{t_C} \frac{1}{N_{S_t}} \sum_{i=1}^{N_{S_t}} |\hat{n}_i^{C_{S_t}}|. \tag{15}$$

Equation (15) measures how frequently or strongly dictionary directions appear across categories similar to C_u. For each layer in W^+, the top-t_A directions with the highest values in $\overline{|n|}^{C_u}$ are selected to form the adaptive category-irrelevant dictionary $A_{C_u} \in \mathbb{R}^{18\times512\times t_A}$.

Assuming that the sparse representations of similar categories follow a Gaussian distribution, the model estimates $\mathcal{N}(\mu_{C_u}, \Sigma_{C_u})$ from the selected sparse codes. A sampled sparse code $\tilde{n}_j$ is used to generate a new image:

$$X_j^{C_u} = G\left(\hat{e}^{C_u} + \alpha A_{C_u}\tilde{n}_j, H(F_{k,i}, \alpha A_{C_u}\tilde{n}_j)\right). \tag{16}$$

Equation (16) defines the final FeatureSAGE synthesis step. The term $\hat{e}^{C_u}$ preserves category-relevant structure, $A_{C_u}\tilde{n}_j$ introduces category-irrelevant variation, α controls edit intensity, and $H(\cdot)$ provides the edited feature tensor used by the generator.

3.4.6 Feature Editing Decoder Block

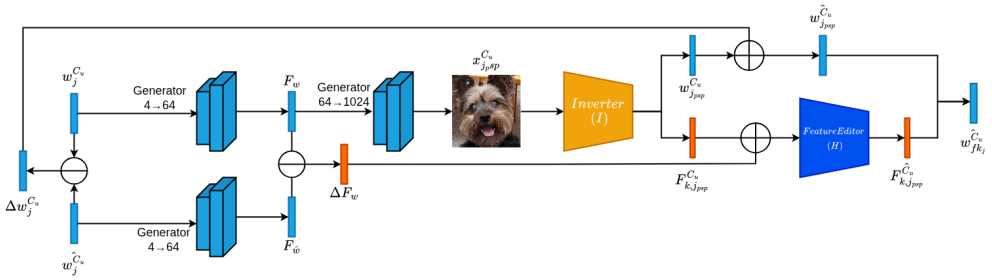

Fig. 10 Feature Editing Decoder Block

Figure 10 shows the Feature Editing Decoder Block. The decoder receives the original latent representation, the edited latent representation from the adaptive editing block, and the intermediate feature tensor predicted by the StyleFeatureEditor Inverter. In this study, the feature tensor injected into the decoder is the layer-9 StyleGAN2 feature tensor, denoted as $F_{k,i}$ when $k = 9$.

The feature displacement used by the Feature Editor is computed as

$$\Delta F_w = F_w - F_{\hat{w}}. \tag{17}$$

Equation (17) compares the generator feature tensor of the original latent code with the generator feature tensor of the edited latent code. This displacement tells $H(\cdot)$ which feature-level details must be adjusted so that the image follows the latent edit while preserving reconstruction detail.

The decoder therefore modifies the intermediate feature representation rather than replacing the full image. The edited feature tensor $H(F_{k,i}, \Delta F_w)$ is injected back into StyleGAN2 at layer 9, while the edited W^+ latent controls the global semantic direction of synthesis. This combination allows the generator to transform the input features into an output image with improved detail preservation and controlled attribute variation.

3.4.7 Training Strategy

In the training stage, StyleGAN2 is first trained with images from the seen categories. The StyleFeatureEditor Inverter and the pSp encoder are also trained. To reduce resource consumption, the proposed framework is trained in two phases.

3.4.8 Phase 1: Attribute Factorization Training

Phase 1 trains the attribute factorization block for disentangled sampling of category-irrelevant directions. The loss functions used in this phase are as follows:

- **Sparsity Loss (L_{sparse}):** This loss encourages each sparse code to activate only a few dictionary directions:

$$L_{\text{sparse}} = \|\sigma(\theta_0 n_i - \theta_1)\|_1. \tag{18}$$

  In Equation (18), $\sigma(\cdot)$ is the sigmoid function, n_i is the sparse code, and θ_0 and θ_1 are hyperparameters that control sparsity. The sigmoid compresses the MLP output into a stable range before the L_1 penalty approximates the sparsity behavior of the L_0 objective in Equation (10).
- **Reconstruction Loss (L_{rec}):** This loss keeps the edited image close to the original input:

$$L_{\text{rec}} = \|G(e^{C_s} + An_i) - x_i^{C_s}\|_2. \tag{19}$$

  In Equation (19), e^{C_s} is the class-center latent for the seen category, An_i is the attribute offset, $G(\cdot)$ is the StyleGAN2 generator, and $x_i^{C_s}$ is the original image. The loss anchors editing so that category-irrelevant changes do not destroy the image content.
- **Orthogonality Loss (L_{orth}):**
  This loss separates category-irrelevant directions from category-relevant directions:

$$L_{\text{orth}} = \|B^T A\|_F^2. \tag{20}$$

  In Equation (20), B is the category-relevant embedding matrix, A is the category-irrelevant dictionary, and $\|\cdot\|_F^2$ is the squared Frobenius norm. The mathematical basis is orthogonality: directions in A should have minimal projection onto the category-relevant subspace B so that edits preserve class identity.

  The total attribute factorization loss is

$$L_{\text{af}} = L_{\text{rec}} + \lambda_1 L_{\text{orth}} + \lambda_2 L_{\text{sparse}}, \tag{21}$$

where λ_1 and λ_2 weight the orthogonality and sparsity terms. Equation (21) balances reconstruction fidelity, disentanglement, and sparse semantic editing [13].

Optimization Algorithm

Training is done using the Adam optimizer with the same configuration as [13]: a fixed learning rate of 1×10^{-4}, $\lambda_1 = 5 \times 10^{-4}$, $\lambda_2 = 5 \times 10^{-3}$, $\theta_0 = 5 \times 10^{-1}$, and $\theta_1 = -1$.

3.4.9 Phase 2: Feature Editor Training

Phase 2 trains the Feature Editor H using editing pairs generated with a pretrained pSp encoder. Following [4], the model generates an image pair $(X, \hat{X})$: pSp takes input image X and predicts w, which is edited by a sampled direction using the adaptive editing method from SAGE. This process allows the Feature Editor to learn feature changes associated with category-irrelevant attributes. To avoid degrading output quality when training only on synthetic edits, H is also trained with a classical inversion task where the edit displacement is set to zero.

The editing loss is defined as

$$L_{\text{edit}} = L_2 + \lambda_{\text{lpips}} L_{\text{lpips}}. \tag{22}$$

The inversion loss uses the same reconstruction and perceptual terms:

$$L_{\text{inv}} = L_2 + \lambda_{\text{lpips}} L_{\text{lpips}}. \tag{23}$$

Equations (22) and (23) combine pixel-level reconstruction through L_2 with perceptual similarity through LPIPS. The total Feature Editor loss is

$$L_{\text{fe}} = L_{\text{edit}}(X, \hat{X}) + L_{\text{inv}}(X, \hat{X}). \tag{24}$$

Equation (24) jointly trains the Feature Editor to handle both edited outputs and identity-preserving inversion outputs.

The model is trained using the Adam optimizer with a fixed learning rate of 1×10^{-4} for 15,000 iterations on a single V100 16 GB vGPU. Mixed precision is enabled to improve memory efficiency due to multiple generator passes. The encoder is optimized using a weighted combination of reconstruction and perceptual losses with $\lambda_{l2} = 10$ and $\lambda_{\text{lpips}} = 0.8$, with an additional LPIPS scaling factor of 0.5. The model is initialized with a fixed length of 100 for the sparse dictionary A.

3.5 Evaluation Protocol

3.5.1 Evaluation Metrics

The models are evaluated using quantitative metrics for reconstruction quality, distributional alignment, and perceptual variation under the one-shot test-time setup. The same metric implementations and evaluation parameters are used for AGE, SAGE, and FeatureSAGE.

1. **Fréchet Inception Distance (FID)**

   Fréchet Inception Distance (FID) is used to evaluate the distributional similarity between generated images and real images [18, 39]. Instead of comparing images

directly at the pixel level, FID compares high-level features extracted from a pretrained Inception-v3 network. This makes the metric suitable for measuring both image fidelity and sample diversity in few-shot image generation [5]. The real and generated feature distributions are modeled as multivariate Gaussian distributions, and FID is computed as

$$\mathrm{FID} = \|\mu_r - \mu_g\|_2^2 + \mathrm{Tr}\left(\Sigma_r + \Sigma_g - 2(\Sigma_r\Sigma_g)^{1/2}\right). \tag{25}$$

In Equation (25), μ_r and Σ_r are the mean and covariance of the Inception features of real images, while μ_g and Σ_g are the mean and covariance of the Inception features of generated images. The term $\|\mu_r - \mu_g\|_2^2$ measures the squared Euclidean distance between feature means, $\mathrm{Tr}(\cdot)$ is the trace operator, and $(\Sigma_r\Sigma_g)^{1/2}$ is the matrix square root of the covariance product.

Equation (25) computes the Fréchet, or 2-Wasserstein, distance between the two Gaussian feature distributions [18]. A lower FID indicates that generated images are closer to the real image distribution in terms of quality and diversity. An FID of 0 indicates perfect distributional agreement.

2. **Learned Perceptual Image Patch Similarity (LPIPS)**

Learned Perceptual Image Patch Similarity (LPIPS) evaluates perceptual similarity by comparing deep feature activations from a pretrained neural network, such as VGG or AlexNet, rather than comparing raw pixels [43]. The LPIPS score between images x and x' is computed as

$$\begin{aligned}\mathrm{LPIPS}(x, x') = \sum_l \frac{1}{H_l W_l} \sum_{h=1}^{H_l} \sum_{w=1}^{W_l} \Big\| w_l \odot \\ \left(\hat{y}^l_{hw}(x) - \hat{y}^l_{hw}(x')\right) \Big\|_2^2.\end{aligned} \tag{26}$$

In Equation (26), $\hat{y}^l_{hw}(x)$ is the unit-normalized feature vector of image x at layer l and spatial position (h, w), w_l is the learned channel-weight vector for layer l, and H_l and W_l are the height and width of the feature map at layer l.

LPIPS can be interpreted according to its evaluation context. For paired reconstruction, lower LPIPS indicates stronger perceptual similarity to the reference image. For generated samples in few-shot image generation, LPIPS may also be used to describe perceptual diversity among outputs; in that context, higher LPIPS indicates greater variation between generated images. This study reports LPIPS alongside FID so that improvements in distributional quality can be evaluated together with perceptual variation.

3.5.2 Baselines for Comparison

To contextualize the performance of the proposed few-shot image generation model, the study compares FeatureSAGE against two editing-based baseline methods:

- **AGE** [12] – Leverages latent-space attribute editing on pretrained GANs to produce novel-class images without retraining by disentangling class-relevant and category-irrelevant features.

- **SAGE** [13] – Builds on AGE by aggregating support images to create stable latent embeddings and by injecting class-consistent diversity through attribute mixing.

3.6 Expected Outcome

Based on the research design, FeatureSAGE is expected to improve latent code retrieval from real-world images by replacing the pSp-based inversion path of SAGE with the StyleFeatureEditor Inverter and Feature Editor. The expected technical effect is stronger reconstruction and more stable editing because the framework represents each image using both W^+ latent codes and intermediate feature tensors. The W^+ code supports semantic attribute manipulation, while the feature tensor preserves fine visual details that may be weakened by latent-only inversion.

The framework is evaluated on the FUNIT Animal Faces and 102 Flowers datasets under the one-shot setting. Performance is assessed using Fréchet Inception Distance (FID) for distributional alignment and Learned Perceptual Image Patch Similarity (LPIPS) for perceptual variation. These metrics are used consistently across AGE, SAGE, and FeatureSAGE to determine whether the proposed inversion and feature-editing changes improve generated image quality without removing useful output diversity.

The expected measurable outcomes are as follows:

1. Lower FID, reflecting improved visual realism and closer distributional alignment between generated and real images.
2. Comparable LPIPS, reflecting that improvements in FID should not come from collapsing outputs into overly similar samples.

Together, these outcomes support the study's goal of evaluating whether feature-level inversion and editing can strengthen SAGE for few-shot image generation and generative data augmentation in resource-limited settings.

4 Results and Discussion

This chapter presents the experimental results of the proposed FeatureSAGE framework and compares its performance with AGE and SAGE on one-shot image generation tasks. The experiments are conducted on two datasets, FUNIT Animal Faces and 102 Flowers, using a single reference image per target class. The reported evaluation is performed at 1024×1024 resolution. Image quality and distributional alignment are measured using Fréchet Inception Distance (FID), while Learned Perceptual Image Patch Similarity (LPIPS) is reported to describe perceptual variation among generated outputs. The comparison evaluates whether replacing the pSp-based SAGE inversion path with the StyleFeatureEditor Inverter and Feature Editor improves few-shot generation quality while maintaining usable edit diversity.

Table 1 Quantitative comparison of AGE, SAGE, and the proposed FeatureSAGE across Flowers and Animal Faces datasets at 1024×1024 resolution. Lower FID indicates better distributional alignment; LPIPS is reported as a perceptual variation measure.

Resolution	Model	Flowers		Animal Faces	
		FID ↓	LPIPS	FID ↓	LPIPS
1024	AGE (Baseline)	61.57	0.4108	62.13	0.5033
1024	SAGE (Baseline)	54.17	0.4528	60.92	0.4867
1024	**FeatureSAGE (Ours)**	45.60	0.4209	49.60	0.4811

4.1 Quantitative Results

4.1.1 Quantitative Comparison Results between FeatureSAGE and Baselines

4.2 Analysis of Quantitative Results

This section analyzes the quantitative performance trends observed in Table 1, focusing on how the proposed FeatureSAGE model compares against AGE and SAGE baselines across the Flowers and Animal Faces datasets at 1024×1024 resolution using FID and LPIPS metrics.

4.2.1 102 Flowers

On a high-resolution setting, 1024×1024, FeatureSAGE obtains an FID of 45.60, which corresponds to a 15.8% improvement over SAGE (54.17) and a 25.9% improvement over AGE (61.57). This substantial FID reduction demonstrates improved distributional alignment with the real flower images in the Inception feature space. In terms of perceptual diversity, FeatureSAGE achieves an LPIPS of 0.4209, compared to SAGE (0.4528) and AGE (0.4108). This represents a 7.0% decrease relative to SAGE and a 2.5% increase relative to AGE. The LPIPS values across all three methods range from 0.4108 to 0.4528, indicating that perceptual diversity remains relatively consistent across the approaches. The results show that FeatureSAGE achieves FID improvements of 15.8–25.9% while maintaining LPIPS within 7.0% of the baseline methods.

4.2.2 Animal Faces

On the Animal Faces HQ dataset at 1024×1024, FeatureSAGE achieved an FID of 49.60, outperforming SAGE (60.92) by 18.6% and AGE (62.13) by 20.2%. This reduction in FID demonstrates FeatureSAGE's superior distributional alignment with the real data distribution in the Inception feature space. In terms of perceptual diversity, FeatureSAGE achieves an LPIPS of 0.4811, compared to SAGE (0.4867) and AGE (0.5033). This represents a 1.2% decrease relative to SAGE and a 4.4% decrease relative to AGE. The LPIPS differences across all three methods are relatively small, with values ranging from 0.4811 to 0.5033, indicating comparable levels of perceptual

diversity in the VGG feature space. The results show that FeatureSAGE achieves substantial FID improvements, 18.6–20.2%, while maintaining LPIPS values within 4.4% of the baseline methods.

4.3 Qualitative Evaluation

4.3.1 Visual Comparison Across Models

Figures 11–14 present the visual comparison of AGE, SAGE, and the proposed FeatureSAGE model.

Fig. 11 Comparison across AGE, SAGE, and the proposed FeatureSAGE model under one-shot setting, where a single reference image is used to condition the model during inference.

Fig. 12 Comparison across AGE, SAGE, and the proposed FeatureSAGE model under one-shot setting with a different image input.

Fig. 13 Comparison across AGE, SAGE, and the proposed FeatureSAGE model on the Animal Faces dataset under one-shot setting. Each model generates two outputs per input image to demonstrate edit diversity and semantic consistency.

Fig. 14 Comparison across AGE, SAGE, and the proposed FeatureSAGE model under one-shot setting with a different dog input image. Each model generates two outputs per input to illustrate diversity and semantic consistency.

4.3.2 Visual Observations

Flowers

Figures 11 and 12 describe the qualitative analysis of the results of the edited flowers based on the baseline models AGE and SAGE, and the proposed model FeatureSAGE. The models are compared with regard to their ability to maintain the colors properly, the sharpness of the images, the level of contrast, and realism. As shown in Figure 11, the proposed model performs well in the retention of the pink petal colors from the input images. It also performs much better based on the level of semantic consistency. Note that the proposed model also fails to represent the combined red and yellow colors of the pistil from the input images. On the other hand, the proposed model performs much better than both the input images in the sharpness of the petal. Although there are inconsistencies from the input images, the proposed model also performs much better. In addition, the level of contrast in the proposed model results in the sharpest flower among all the models.

Further, the advantages of FeatureSAGE are highlighted by Figure 12. The sharpness in petal edges is better retained by the model compared to AGE and SAGE, which allows for a more photo-realistic and organic rendering. More varied outputs have also resulted from FeatureSAGE; for instance, in one generated image, the flower is presented with a slight tilt, reflecting strong attribute manipulation. The model maintains higher clarity, with balanced contrast and natural color distribution, while AGE slightly looks pale with reduced contrast in areas, which is somewhat settled by SAGE as it balances the contrast. Overall, the generated flower images produced by FeatureSAGE are the sharpest, most realistic, and visually consistent as compared to the other models being tested.

Animal Faces

Figure 13 provides a comparison of the outputs produced by the AGE, SAGE, and proposed FeatureSAGE model on the Animal Faces data set using a picture of a dog as input. As observed, the outputs produced by the models have managed to maintain the color and texture of the input data, implying success by these models in maintaining basic characteristics produced by each model. Considering the clarity and image properties, all the outputs produced by these models can be considered similar, since there is minimal blurring effect and texture preservation on the fur of the input image. Consistency and overall properties, on the other hand, differ significantly, since there is more diversity apparent in the outputs produced by the AGE and SAGE models, with a slight difference in orientation between the two outcomes produced by each model. The more consistent results produced by the proposed FeatureSAGE model, on the contrary, demonstrate success toward maintaining integrity while providing semantic edits. Also, there are some minor enhancements in the realism of the output, whereby the output from FeatureSAGE shows slightly more coherent shading and blending of the fur patterns. This indicates that although all these models demonstrate the ability to sustain basic attributes like color and texture, the FeatureSAGE model has the best consistency in output.

Figure 14 shows similar behavior to the third image input for FeatureSAGE, where it maintains consistency in terms of pose within the results, showing that it does not

have great variability compared to AGE and SAGE, which have more diverse orientations. At the same time, it may be noted that FeatureSAGE is better at maintaining consistency of shape and structure for eyes, thus improving realism. It is also observed that the results from FeatureSAGE have better clarity and contrast, improving image quality, particularly for fur texture and shading. On the other hand, it may be noted that AGE performs best in terms of diversity among all models, particularly for pose variation. Yet, it is observed that this diversity may also decrease image clarity.

4.4 Failure Cases and Limitations

During experimentation and evaluation of the FeatureSAGE framework, several limitations were observed:

1. **Dependence on pretrained StyleGAN2.** FeatureSAGE uses a pretrained StyleGAN2 generator and therefore remains limited to domains where a suitable generator is available. If the base generator does not adequately represent certain visual categories or variations, those limitations can carry over to the generated results. This dependence also means that biases or weaknesses in the pretrained latent space may affect attribute editing behavior.
2. **Computational overhead.** Adding the StyleFeatureEditor Inverter and Feature Editor increases computation relative to the base SAGE setup. The additional inversion, feature extraction, and feature-injection steps improve reconstruction detail, but they also require more GPU memory and increase processing time per operation. This trade-off is important when scaling to larger datasets or time-sensitive applications.

These limitations indicate that FeatureSAGE improves image quality and stability at the cost of a heavier processing pipeline. Future work should therefore examine computational optimization, alternative base generators, and multi-level feature editing strategies while preserving the same goal of stable few-shot attribute group editing.

4.5 Chapter Summary

The results show that FeatureSAGE achieved the lowest FID on both evaluated datasets. On 102 Flowers, FeatureSAGE obtained an FID of 45.60, improving over SAGE by 15.8% and over AGE by 25.9%. On Animal Faces, FeatureSAGE obtained an FID of 49.60, improving over SAGE by 18.6% and over AGE by 20.2%. These outcomes indicate stronger distributional alignment with the real image data after integrating the StyleFeatureEditor Inverter and Feature Editor into the SAGE pipeline. LPIPS remained within a narrow range relative to the baselines, with 0.4209 on Flowers and 0.4811 on Animal Faces, indicating that the FID gains did not remove perceptual variation from the generated outputs. Qualitative comparisons further show that FeatureSAGE produced sharper flower structures, more coherent color and contrast, and more stable animal-face structure, although the method still carries computational overhead and remains dependent on the pretrained StyleGAN2 generator.

5 Conclusion and Recommendations

5.1 Summary of Findings

This study evaluated whether integrating the StyleFeatureEditor Inverter and Feature Editor into SAGE improves few-shot image generation under a one-shot setting. The findings are summarized as follows:

5.1.1 Integration of the StyleFeatureEditor Inverter

FeatureSAGE replaces the pSp-based inversion path of SAGE with the StyleFeatureEditor Inverter, allowing the framework to represent each image through both W^+ latent codes and intermediate feature tensors. This directly addresses the research problem concerning inversion fidelity and detail preservation. At 1024×1024 resolution, FeatureSAGE achieved an FID of 45.60 on 102 Flowers and 49.60 on Animal Faces.

5.1.2 Use of the Feature Editor for feature-level detail preservation

The Feature Editor modifies intermediate feature tensors together with the edited latent code. This supports the decoder block by preserving local detail while allowing semantic attribute edits. Qualitative comparisons show sharper petal structures, stronger contrast, and more coherent animal-face structure for FeatureSAGE compared with AGE and SAGE.

5.1.3 Quantitative improvement over AGE and SAGE

FeatureSAGE obtained the best FID score on both evaluated datasets. On 102 Flowers, it improved FID by 15.8% over SAGE and 25.9% over AGE. On Animal Faces, it improved FID by 18.6% over SAGE and 20.2% over AGE. These results indicate stronger distributional alignment between generated and real images.

5.1.4 Preservation of perceptual variation

The reported LPIPS values remained close to the baselines. FeatureSAGE obtained an LPIPS of 0.4209 on 102 Flowers, compared with 0.4528 for SAGE and 0.4108 for AGE. On Animal Faces, FeatureSAGE obtained an LPIPS of 0.4811, compared with 0.4867 for SAGE and 0.5033 for AGE. These values indicate that the FID improvements were achieved while maintaining comparable perceptual variation across generated outputs.

5.2 Limitations

Though FeatureSAGE holds promise in attribute group editing and limited example image synthesis, some constraints observed during testing indicate that there is scope for further research. The first is the reliance on a pre-trained StyleGAN2 generator, which restricts the application of this tool only to domains where such generators have been developed, and any biases in the model may have been passed on to the output. The second is the potential performance impact of integrating the StyleFeatureEditor with the Feature Editor, which increases computational requirements. Future

research could focus on other base models, incorporate a multi-level feature approach to improve consistency between edits, optimize computation for more general or faster models, and extend few-shot adaptation to work well even on entirely new classes.

5.3 Recommendations

Future research should focus on reducing the computational overhead introduced by the StyleFeatureEditor Inverter and Feature Editor. Since FeatureSAGE requires additional feature extraction, feature editing, and feature injection steps, optimization should target faster inference and lower GPU memory usage while preserving the reconstruction and editing advantages observed in this study.

Further work may also examine alternative base generators beyond StyleGAN2. Because FeatureSAGE depends on the availability and quality of a pretrained generator, testing other generator backbones could clarify how much of the framework's performance depends on the StyleGAN2 latent and feature spaces. Another direction is to study multi-level feature editing, where feature tensors from more than one generator layer are considered, to determine whether consistency between fine details and higher-level structure can be improved without increasing instability.

5.4 Conclusion

This study addressed the problem of improving few-shot image generation in SAGE by replacing its pSp-based inversion path with a StyleFeatureEditor Inverter and Feature Editor. The original research problem asked whether improved inversion and feature-level editing could preserve image details, support stable semantic edits, and improve generated image quality for novel classes in the Flowers and Animal Faces datasets. Based on the reported results, the answer is affirmative: FeatureSAGE achieved lower FID than both AGE and SAGE on the two evaluated datasets while maintaining comparable LPIPS values.

The first objective, integrating StyleFeatureEditor into SAGE, was achieved by incorporating the StyleFeatureEditor Inverter and Feature Editor into the FeatureSAGE architecture. The second objective, evaluating reconstruction and generation quality using FID and LPIPS, was addressed through the quantitative evaluation at 1024×1024 resolution. FeatureSAGE achieved an FID of 45.60 and LPIPS of 0.4209 on 102 Flowers, and an FID of 49.60 and LPIPS of 0.4811 on Animal Faces. The third objective, benchmarking against AGE and SAGE, was also achieved: FeatureSAGE improved FID by 15.8% over SAGE and 25.9% over AGE on 102 Flowers, and by 18.6% over SAGE and 20.2% over AGE on Animal Faces.

Overall, the study contributes an enhanced SAGE-based framework that combines latent-space attribute editing with feature-level detail preservation. The findings show that this integration improves distributional alignment and qualitative image coherence in one-shot generation while retaining perceptual variation. However, the framework remains dependent on a pretrained StyleGAN2 generator and requires greater computation because of the additional inversion and feature-editing stages.

Declarations

Funding Not applicable.
Conflict of interest The authors declare that they have no conflict of interest.

References

[1] Abdal R, Zhu P, Mitra NJ, et al (2021) Styleflow: Attribute-conditioned exploration of stylegan-generated images using conditional continuous normalizing flows. ACM Transactions on Graphics 40(3):1–21. https://doi.org/10.1145/3447648, URL http://dx.doi.org/10.1145/3447648

[2] Alaluf Y, Patashnik O, Cohen-Or D (2021) Restyle: A residual-based stylegan encoder via iterative refinement. URL https://arxiv.org/abs/2104.02699, arXiv:2104.02699

[3] Bermano AH, Gal R, Alaluf Y, et al (2022) State-of-the-art in the architecture, methods and applications of stylegan. URL https://arxiv.org/abs/2202.14020, arXiv:2202.14020

[4] Bobkov D, Titov V, Alanov A, et al (2024) The devil is in the details: Stylefeatureeditor for detail-rich stylegan inversion and high quality image editing. URL https://arxiv.org/abs/2406.10601, arXiv:2406.10601

[5] Borji A (2022) Pros and cons of gan evaluation measures. Computer Vision and Image Understanding 215:103329

[6] Cao Y, Gong S (2024) Few-shot image generation by conditional relaxing diffusion inversion. URL https://arxiv.org/abs/2407.07249, arXiv:2407.07249

[7] Chen X, Duan Y, Houthooft R, et al (2016) Infogan: Interpretable representation learning by information maximizing generative adversarial nets. URL https://arxiv.org/abs/1606.03657, arXiv:1606.03657

[8] Chowdhury A, Jiang M, Chaudhuri S, et al (2021) Few-shot image classification: Just use a library of pre-trained feature extractors and a simple classifier. URL https://arxiv.org/abs/2101.00562, arXiv:2101.00562

[9] Clouatre L, Demers M (2019) Figr: Few-shot image generation with reptile. URL https://arxiv.org/abs/1901.02199, arXiv:1901.02199

[10] Cohen G, Giryes R (2022) Generative adversarial networks. URL https://arxiv.org/abs/2203.00667, arXiv:2203.00667

[11] Creswell A, White T, Dumoulin V, et al (2018) Generative adversarial networks: An overview. IEEE Signal Processing Magazine 35(1):53–65. https://doi.org/10.1109/msp.2017.2765202, URL http://dx.doi.org/10.1109/MSP.2017.2765202

[12] Ding G, Han X, Wang S, et al (2022) Attribute group editing for reliable few-shot image generation. URL https://arxiv.org/abs/2203.08422, arXiv:2203.08422

[13] Ding G, Han X, Wang S, et al (2023) Stable attribute group editing for reliable few-shot image generation. URL https://arxiv.org/abs/2302.00179, arXiv:2302.00179

[14] Goodfellow I (2017) Nips 2016 tutorial: Generative adversarial networks. URL https://arxiv.org/abs/1701.00160, arXiv:1701.00160

[15] Goodfellow IJ, Pouget-Abadie J, Mirza M, et al (2014) Generative adversarial networks. URL https://arxiv.org/abs/1406.2661, arXiv:1406.2661

[16] Gupta P, Hayat M, Dhall A, et al (2024) Conditional distribution modelling for few-shot image synthesis with diffusion models. URL https://arxiv.org/abs/2404.16556, arXiv:2404.16556

[17] Harkonen E, Hertzmann A, Lehtinen J, et al (2020) Ganspace: Discovering interpretable gan controls. URL https://arxiv.org/abs/2004.02546, arXiv:2004.02546

[18] Heusel M, Ramsauer H, Unterthiner T, et al (2017) Gans trained by a two time-scale update rule converge to a local nash equilibrium. In: Advances in Neural Information Processing Systems

[19] Hong Y, Niu L, Zhang J, et al (2022) Deltagan: Towards diverse few-shot image generation with sample-specific delta. URL https://arxiv.org/abs/2009.08753, arXiv:2009.08753

[20] Karras T, Aila T, Laine S, et al (2018) Progressive growing of gans for improved quality, stability, and variation. URL https://arxiv.org/abs/1710.10196, arXiv:1710.10196

[21] Karras T, Laine S, Aila T (2019) A style-based generator architecture for generative adversarial networks. URL https://arxiv.org/abs/1812.04948, arXiv:1812.04948

[22] Karras T, Laine S, Aittala M, et al (2020) Analyzing and improving the image quality of stylegan. URL https://arxiv.org/abs/1912.04958, arXiv:1912.04958

[23] Li J, Li J, Zhang H, et al (2023) Preim3d: 3d consistent precise image attribute editing from a single image. URL https://arxiv.org/abs/2304.10263, arXiv:2304.10263

[24] Liu H, Song Y, Chen Q (2023) Delving stylegan inversion for image editing: A foundation latent space viewpoint. URL https://arxiv.org/abs/2211.11448, arXiv:2211.11448

[25] Liu MY, Huang X, Mallya A, et al (2019) Few-shot unsueprvised image-to-image translation. In: arxiv

[26] Liu MY, Huang X, Mallya A, et al (2019) Few-shot unsupervised image-to-image translation. URL https://arxiv.org/abs/1905.01723, arXiv:1905.01723

[27] Melnik A, Miasayedzenkau M, Makaravets D, et al (2024) Face generation and editing with stylegan: A survey. IEEE Transactions on Pattern Analysis and Machine Intelligence 46(5):3557–3576. https://doi.org/10.1109/tpami.2024.3350004, URL http://dx.doi.org/10.1109/TPAMI.2024.3350004

[28] Mo S, Cho M, Shin J (2020) Freeze the discriminator: a simple baseline for fine-tuning gans. URL https://arxiv.org/abs/2002.10964, arXiv:2002.10964

[29] Nilsback ME, Zisserman A (2008) Automated flower classification over a large number of classes. In: Indian Conference on Computer Vision, Graphics and Image Processing

[30] Ojha U, Li Y, Lu J, et al (2021) Few-shot image generation via cross-domain correspondence. URL https://arxiv.org/abs/2104.06820, arXiv:2104.06820

[31] Patashnik O, Wu Z, Shechtman E, et al (2021) Styleclip: Text-driven manipulation of stylegan imagery. URL https://arxiv.org/abs/2103.17249, arXiv:2103.17249

[32] Richardson E, Alaluf Y, Patashnik O, et al (2021) Encoding in style: a stylegan encoder for image-to-image translation. URL https://arxiv.org/abs/2008.00951, arXiv:2008.00951

[33] Salehi P, Chalechale A, Taghizadeh M (2020) Generative adversarial networks (gans): An overview of theoretical model, evaluation metrics, and recent developments. URL https://arxiv.org/abs/2005.13178, arXiv:2005.13178

[34] Shen Y, Zhou B (2021) Closed-form factorization of latent semantics in gans. URL https://arxiv.org/abs/2007.06600, arXiv:2007.06600

[35] Shen Y, Gu J, Tang X, et al (2020) Interpreting the latent space of gans for semantic face editing. URL https://arxiv.org/abs/1907.10786, arXiv:1907.10786

[36] Song Y, Wang T, Mondal SK, et al (2022) A comprehensive survey of few-shot learning: Evolution, applications, challenges, and opportunities. URL https://arxiv.org/abs/2205.06743, arXiv:2205.06743

[37] Tov O, Alaluf Y, Nitzan Y, et al (2021) Designing an encoder for stylegan image manipulation. URL https://arxiv.org/abs/2102.02766, arXiv:2102.02766

[38] Voynov A, Babenko A (2020) Unsupervised discovery of interpretable directions in the gan latent space. URL https://arxiv.org/abs/2002.03754, arXiv:2002.03754

[39] Wang Y, Li C, Loy CC (2023) Attribute group editing for reliable few-shot image generation. arXiv preprint arXiv:220308422

[40] Wu Z, Lischinski D, Shechtman E (2020) Stylespace analysis: Disentangled controls for stylegan image generation. URL https://arxiv.org/abs/2011.12799, arXiv:2011.12799

[41] Xia W, Zhang Y, Yang Y, et al (2022) Gan inversion: A survey. URL https://arxiv.org/abs/2101.05278, arXiv:2101.05278

[42] Zhang R, Isola P, Efros AA, et al (2018) The unreasonable effectiveness of deep features as a perceptual metric. URL https://arxiv.org/abs/1801.03924, arXiv:1801.03924

[43] Zhang R, Isola P, Efros AA, et al (2018) The unreasonable effectiveness of deep features as a perceptual metric. In: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), pp 586–595

[44] Zhang R, Huang G, Huo Y, et al (2024) Tage: Trustworthy attribute group editing for stable few-shot image generation. URL https://arxiv.org/abs/2410.17855, arXiv:2410.17855

[45] Zhao Y, Ding H, Huang H, et al (2023) A closer look at few-shot image generation. URL https://arxiv.org/abs/2205.03805, arXiv:2205.03805